Динамическое дерево Хаффмана с прогнозированием критичности состояния для оптимизации очереди приема пациентов в отделении неотложной помощи

Москва – 2026

СОДЕРЖАНИЕ

Введение 3

Глава 1. Анализ существующих методов и алгоритмов 6

1.1. Медицинский триаж как задача приоритезации пациентов 6

1.2. Классические алгоритмы приоритетных очередей 7

1.3. Классическое дерево Хаффмана 11

Глава 2. Алгоритм инверсного дерева Хаффмана с прогнозированием критичности состояния 14

2.1. Концепция инверсии алгоритма Хаффмана 14

2.2. Математическая модель прогнозирования критичности и веса пациента 15

2.3. Алгоритм динамического перестроения дерева 20

2.4. Структуры данных для реализации на C++ 23

Глава 3. Программная реализация и тестирование 25

3.1. Программная реализация 25

3.2. Тестирование 27

Заключение 29

Введение

Актуальность

С апреля 2025 года в Москве действуют шесть новых флагманских центров оказания экстренной медицинской помощи. Каждый рассчитан на приём до 200 человек в сутки и работает по системе триажа, учитывающую жизненные показатели пациента при поступлении [1].

Триаж — распределение пациентов по приоритетам помощи — остаётся преимущественно статической процедурой: медсестра один раз присваивает категорию срочности, и дальше она не пересматривается. В международном многоцентровом исследовании Wuerz и соавторов (2017), охватившем более 5,3 млн обращений, точность ESI для пациентов высокой и низкой остроты составила лишь 44% и 54% соответственно, а чувствительность в выявлении критически больных — 66% [4].

Но даже правильно присвоенный приоритет не учитывает двух факторов: состояние пациента может ухудшаться прямо в очереди, а один ресурсоёмкий случай способен заблокировать помощь нескольким более лёгким. Медсёстры сами признают, что ESI не даёт инструментов для реакции на такую динамику [5]. Классические алгоритмы приоритетных очередей — бинарная куча, STL priority_queue — устроены так же: порядок поддерживается по текущим значениям, а прогнозировать изменения они не умеют.

Отсюда и возникает задача настоящей работы: нужна структура данных, способная перестраивать очередь на основе предсказанных изменений состояния, а не только фактических.

Цель и задачи исследования

Цель работы — разработка и программная реализация алгоритма динамического управления очередью пациентов на основе инверсного дерева Хаффмана с прогнозированием изменения критичности состояния для максимизации числа спасённых жизней в условиях ограниченных медицинских ресурсов.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести анализ существующих методов медицинской сортировки и классических алгоритмов приоритетных очередей, выявить их ограничения при работе с динамически меняющимися состояниями пациентов.

2. Исследовать классический алгоритм Хаффмана и обосновать необходимость его модификации с инверсией весов для применения в задаче приоритетной очереди.
3. Разработать математическую модель изменения критичности состояния пациента на основе дифференциальных уравнений показательного роста или убывания.
4. Спроектировать алгоритм инверсного дерева Хаффмана, в котором объединение двух наибольших весов обеспечивает нахождение наиболее критичного пациента в корневой позиции.
5. Разработать комбинаторный алгоритм динамического перестроения дерева при изменении приоритетов с учётом прогнозируемых значений.
6. Реализовать программный модуль на языке C++, моделирующий работу приёмного отделения с использованием разработанного алгоритма
7. Провести вычислительные эксперименты для оценки эффективности предложенного алгоритма в сравнении с классическими аналогами и сформулировать выводы о применимости разработанного алгоритма в реальных медицинских системах.

Объект и предмет исследования

Объект исследования — процесс управления очередью пациентов в отделении неотложной помощи.

Предмет исследования — алгоритмы и структуры данных для динамической приоритезации пациентов с учётом прогноза изменения критичности состояния.

Методы исследования

В работе применяются методы анализа алгоритмов, включая оценку вычислительной сложности по времени и памяти; методы математического моделирования — дифференциальные уравнения для описания динамики изменения состояния пациента; методы комбинаторной оптимизации — инверсное дерево Хаффмана как способ организации приоритетной очереди; методы вычислительного эксперимента — сравнительное тестирование разработанного алгоритма и аналогов на синтетических данных, сгенерированных на основе реальных статистических распределений.

Научная новизна

Научная новизна работы заключается в использовании инверсного дерева Хаффмана (объединение не двух наименьших, а двух наибольших весов) для задачи приоритетной очереди. Разработана комбинаторная процедура

динамического перестроения дерева с учётом прогнозируемых изменений весов. Введена весовая функция, ориентированная на максимизацию общего числа спасённых жизней, учитывающая как оставшееся безопасное время, так и ожидаемую длительность лечения.

Практическая значимость

Разработанный алгоритм может найти применение в системах поддержки принятия решений при медицинской сортировке. Он также может быть использован в симуляционных тренажёрах для обучения персонала отделений неотложной помощи. Кроме того, предложенный подход может быть адаптирован для других областей, где требуется динамическое управление приоритетами с прогнозированием, например, в системах обработки инцидентов или управления вычислительными ресурсами.

Структура работы

Работа состоит из введения, трёх глав, заключения, списка использованных источников и приложения.

В **первой главе** проводится анализ существующих методов медицинской сортировки и классических алгоритмов приоритетных очередей. Приводится оценка их вычислительной сложности, выявляются ограничения, обосновывающие необходимость разработки нового подхода. Глава завершается краткими выводами.

Во **второй главе** разрабатывается авторский алгоритм. Вводится концепция инверсии правил объединения Хаффмана, строится математическая модель прогнозирования критичности на основе дифференциальных уравнений, описывается комбинаторная процедура динамического перестроения дерева, детально рассматриваются используемые структуры данных. Глава также завершается выводами.

В **третьей главе** представляется программная реализация на языке C++. Описывается архитектура модуля, структуры данных, сценарий вычислительных экспериментов. Приводятся и анализируются полученные результаты. В конце главы формулируются выводы по результатам экспериментов.

В **заключении** обобщаются основные результаты работы, формулируются выводы по каждой из поставленных задач и намечаются направления дальнейших исследований.

Глава 1. Анализ существующих методов и алгоритмов

1.1. Медицинский триаж как задача приоритезации пациентов

Постановка задачи. При поступлении пациента в отделение неотложной помощи необходимо определить его место в очереди на обслуживание. Эта процедура называется триажом (Приложение 1). Формально процесс триажа можно представить как функцию, которая каждому пациенту p из множества поступивших P ставит в соответствие категорию срочности c из конечного упорядоченного множества $C = \{c_1, c_2, \ldots, c_k\}$, где c_1 — наивысший приоритет (наиболее критичные пациенты), а c_k — самый низкий.

Категория срочности определяет максимально допустимое время ожидания помощи $T_{max}(c)$ или, в более простых системах, просто порядок обслуживания. Задача системы триажа — обеспечить, чтобы в каждый момент времени на обработку поступал пациент с максимальным текущим приоритетом, а распределение времени ожидания соответствовало заданным нормативам.

В существующей практике эта формальная модель реализуется через клинические шкалы, которые присваивают категорию срочности на основе набора эмпирических правил.

Основные шкалы триажа. На сегодняшний день наиболее распространёнными являются:

1. ESI (Emergency Severity Index) (Приложение 1) — пятиуровневая шкала (от 1 до 5). Алгоритмически она представляет собой дерево решений: медсестра последовательно отвечает на вопросы о стабильности витальных функций (Приложение 1), ожидаемом использовании медицинских ресурсов (Приложение 1) и наличии высокорисковых симптомов [2]. Формально ESI можно описать как функцию $c = f_{ESI}(v, r, s)$, где v — стабильность виталов, r — прогнозируемое количество ресурсов, s — наличие высокорисковых симптомов.
2. CTAS (Canadian Triage and Acuity Scale) (Приложение 1) — пятиуровневая шкала, которая, помимо категории, задаёт жёсткие таргеты (Приложение 1) — максимальное время до осмотра врачом. Формально: $c = f_{CTAS}(vital_signs, chief_complaint)$, и каждому c соответствует $T_{target}(c)$.
3. MTS (Manchester Triage System) (Приложение 1) — пятиуровневая шкала, использующая флау-чарты (Приложение 1) на основе ведущего симптома (боль в груди, одышка, падение и т.п.) [3]. Формально: $c =$

$f_{MTS}(symptom, discriminator)$, где discriminator — ключевой признак (например, частота дыхания, уровень сатурации).

В Приложении 2 приведено сравнение трёх описанных систем триажа по ключевым параметрам: заявленной цели, рекомендуемому времени до осмотра врачом, используемым дискриминаторам и учёту педиатрических пациентов. Таблица адаптирована из систематического обзора Hinson [6], где представлен анализ характеристик различных шкал триажа.

В России единой шкалы триажа нет, внедрение происходит на уровне регионов. В московских флагманских центрах экстренной помощи используется упрощённая трёхуровневая система (красный — жёлтый — зелёный) [1].

Программные решения. В крупных больницах процесс триажа автоматизирован. В Москве применяется модуль «Триаж.Стационар» — часть ЕМИАС (Приложение 1). Медсестра фиксирует жалобы и виталы в системе, которая помогает присвоить категорию срочности и фиксирует время ожидания. Однако при ухудшении пациента переоценка выполняется вручную. В зарубежных клиниках используются EPIC, Cerner, Meditech, которые автоматически рассчитывают ESI, но также не пересматривают приоритет автоматически.

Ограничения статического подхода. Все перечисленные системы реализуют статическую приоритезацию: категория присваивается один раз при поступлении и далее не пересматривается. Это порождает три проблемы.

Первая проблема— неучёт динамики состояния пациента. Пациент может ухудшаться в очереди (при сепсисе, инфаркте), но его категория остаётся прежней. Вторая связана с субъективностью: разные медсёстры трактуют понятие «высокого риска» по-разному и получают разные категории для одного и того же пациента[5]. Кроме того, в исследовании взрослых (5,3 млн случаев) неверная сортировка выявлена примерно в каждом третьем обращении [4].

Можно сделать вывод, что существующие системы не обеспечивают динамической переоценки приоритетов и не учитывают прогноза изменения состояния пациента. Это создаёт потребность в алгоритмическом решении, которое перестраивало бы очередь автоматически при изменении состояния пациентов, или в лучшем случае - на основе прогноза.

1.2. Классические алгоритмы приоритетных очередей

Приоритетная очередь — это абстрактная структура данных, поддерживающая динамическое множество элементов, каждый из которых имеет числовой ключ (приоритет) [7]. В рамках данной структуры набор

элементов может изменяться в процессе работы программы: добавляются новые, удаляются существующие, изменяются их свойства, и количество элементов заранее не известно.

Основные операции, которые должна поддерживать приоритетная очередь:

- **Вставка (push/insert)** — добавление нового элемента с заданным приоритетом.
- **Извлечение максимума (pop/extract_max)** — удаление и возврат элемента с наибольшим приоритетом.
- **Просмотр максимума (top)** — получение элемента с наибольшим приоритетом без его удаления.
- **Изменение приоритета (increase_key / decrease_key)** — обновление значения ключа существующего элемента.

В контексте медицинской сортировки элементами очереди являются пациенты, а приоритетом — категория срочности (или вычисляемый вес). Задача системы — в каждый момент времени передавать врачу пациента с максимальным текущим приоритетом.

Наивные реализации:

1. **Несортированный список:** элементы хранятся в массиве в произвольном порядке.

- **Вставка:** новый элемент добавляется в конец списка. Вставка не требует поддержания порядка, поэтому выполняется за O(1) операций.
- **Извлечение максимума:** необходимо просмотреть все n элементов, чтобы найти максимальный приоритет. Требуется n-1 сравнение в худшем случае. Сложность — O(n).
- **Изменение приоритета:** сначала нужно найти элемент (линейный поиск, O(n)), затем обновить значение (O(1)). Общая сложность — O(n).

2. **Сортированный список.** Альтернатива — поддерживать список постоянно отсортированным по убыванию приоритета.

- **Вставка:** требуется найти позицию для нового элемента. При бинарном поиске — O(log n), но физическая вставка требует сдвига элементов, что в худшем случае даёт O(n).
- **Извлечение максимума:** максимальный элемент находится в начале (или в конце) списка, поэтому извлечение выполняется за O(1).

- **Изменение приоритета:** после изменения приоритета элемент может нарушить порядок. Необходимо удалить его и вставить заново — O(n).

Обе наивные реализации имеют линейную сложность хотя бы для одной из ключевых операций, что делает их неприемлемыми для работы с динамическими множествами большого размера [8].

3. Бинарная куча

Бинарная куча (пирамида, binary heap) — наиболее распространённая структура данных для реализации приоритетной очереди [7]. Она представляет собой полное бинарное дерево, для которого выполняется свойство кучи: приоритет каждого узла не меньше (для max-кучи) приоритетов его потомков.

Хранение. Бинарная куча хранится в массиве без использования явных указателей. Для элемента с индексом i:

- родитель находится по индексу **(i-1)/2** (целочисленное деление);
- левый потомок — **2*i + 1**;
- правый потомок — **2*i + 2**.

Такое представление позволяет эффективно перемещаться по дереву.

Вставка элемента. Новый элемент помещается в конец массива. При этом может нарушиться свойство кучи (новый элемент может оказаться больше родителя). Для восстановления свойства выполняется операция «просеивание вверх» (sift up) : элемент последовательно обменивается с родителем, пока не займёт правильное место в дереве.

Поскольку высота полного бинарного дерева с n элементами равна $\lfloor \log_2 n \rfloor$, количество обменов в худшем случае не превышает O(log n). Следовательно, сложность вставки — O(log n) [7].

В Приложении 3 проиллюстрирован процесс вставки элемента.

Извлечение максимума. Корневой элемент (максимум) сохраняется для возврата. Последний элемент массива перемещается в корень. Затем выполняется операция «просеивание вниз» (sift down): корневой элемент сравнивается с потомками и обменивается с большим из них; процесс повторяется, пока свойство кучи не восстановится. Сложность — O(log n) [8].

Изменение приоритета. Если приоритет элемента увеличивается, он «просеивается вверх»; если уменьшается — «просеивается вниз». В обоих случаях выполняется не более O(log n) обменов.

Таблица 1.

Сравнительная таблица сложностей операций

Операция	Несортированный список	Сортированный список	Бинарная куча
Вставка	O(1)	O(n)	O(log n)
Извлечение максимума	O(n)	O(1)	O(log n)
Изменение приоритета	O(n)	O(n)	O(log n)
Память	O(n)	O(n)	O(n)

Стандартная реализация в C++

В стандартной библиотеке C++ приоритетная очередь реализована как адаптер контейнера std::priority_queue. По умолчанию она использует std::vector в качестве базового контейнера и реализует max-кучу.

Основные методы и их сложность:

- top() — возвращает элемент с наибольшим приоритетом. Выполняется за O(1).
- push(x) — вставляет элемент x за O(log n).
- pop() — удаляет элемент с наибольшим приоритетом за O(log n).

Стандарт C++ гарантирует указанные асимптотические оценки, а в большинстве компиляторов std::priority_queue реализована на основе бинарной кучи [9].

Ограничения классических подходов для задачи триажа

Бинарная куча обеспечивает логарифмическое время выполнения всех операций, что делает её подходящей для динамических приоритетных очередей. Однако для задачи медицинской сортировки с прогнозированием у неё есть два фундаментальных ограничения.

Первое ограничение — отсутствие прогнозирования. Бинарная куча поддерживает порядок на основе текущих значений приоритетов. Если

приоритет пациента изменяется (например, фиксируется ухудшение состояния), требуется явный вызов операции изменения приоритета (O(log n)). Однако в куче нет механизма, который позволял бы учитывать будущие изменения, предсказанные на основе динамики состояния.

Второе ограничение — игнорирование длительности обслуживания. Классическая приоритетная очередь не учитывает ожидаемое время лечения пациента. В задаче максимизации числа спасённых жизней это критично: пациент, требующий длительного лечения, может заблокировать очередь, в то время как несколько пациентов с быстрым лечением могли бы быть спасены.

Бинарная куча и её реализация std::priority_queue являются эффективными структурами для динамической приоритетной очереди при условии, что приоритеты меняются только дискретно (в моменты внешних событий). Однако она не предназначена для работы с прогнозируемыми изменениями и не учитывает длительность обслуживания [7, 8]. Это создаёт потребность в модифицированной структуре данных, способной перестраиваться на основе предсказанных значений весов.

1.3. Классическое дерево Хаффмана

Алгоритм Хаффмана был разработан в 1952 году Дэвидом Хаффманом. Алгоритм предназначен для оптимального префиксного кодирования символов с минимальной избыточностью [11]: символы, встречающиеся чаще, кодируются более короткими кодами, а редкие — более длинными, что обеспечивает максимальную степень сжатия данных.

Принцип работы. Алгоритм строит бинарное дерево, в котором листья соответствуют кодируемым символам, а внутренние узлы — объединению двух поддеревьев. Ключевое правило классического алгоритма Хаффмана: на каждом шаге объединяются два узла с наименьшими весами (частотами встречаемости символов). Родительскому узлу присваивается вес, равный сумме весов объединяемых узлов. Цикл извлечения двух минимальных узлов, создания родительского узла и добавления его в очередь повторяется до тех пор, пока в очереди не останется один узел — корень дерева.

Формальное описание. Пусть дан набор символов $S = \{s_1, s_2, \ldots, s_n\}$ с весами w_i (частотами). Алгоритм:

1. Поместить все символы в приоритетную очередь (min-heap) с ключом w_i.
2. Пока в очереди больше одного узла:
    2.1. Извлечь два узла x и y с наименьшими весами.
    2.2. Создать новый узел z с весом $w_z = w_x + w_y$.

2.3. Сделать x и y потомками z.

2.4. Добавить z в очередь.

3. Оставшийся узел — корень дерева Хаффмана.

Оценка сложности. При использовании приоритетной очереди (бинарной кучи) для извлечения минимальных элементов критическим параметром является количество итераций, которое составляет n-1 (объединение всех узлов). Каждая такая итерация включает две операции извлечения минимума (pop) и одну операцию вставки (push). Каждая из этих операций в бинарной куче имеет сложность $O(\log m)$, где m — текущее количество узлов в очереди. Поскольку m меняется от n до 1, а каждая операция выполняется за логарифм от текущего размера очереди, общая сложность оценивается как $O(n\log n)$.

На первом шаге в очереди n элементов, извлечение двух минимумов и вставка нового узла требуют $O(\log n) + O(\log(n-1)) + O(\log(n-1))$, что в сумме даёт $O(\log n)$. На следующем шаге элементов становится $n-1$, и операции занимают $O(\log(n-1))$, и так далее. Общая сумма логарифмов: $\log n + \log(n-1) + \ldots + \log 1 = \log(n!)$. По формуле Стирлинга $\log(n!) = O(n\log n)$. Следовательно, полное построение дерева Хаффмана выполняется за $O(n\log n)$ [7].

Почему классическое дерево Хаффмана не подходит для очереди пациентов. Несмотря на эффективность в задачах сжатия данных, классический алгоритм Хаффмана неприменим для динамической приоритетной очереди по трём причинам:

1. **Статичность построения.** Дерево строится один раз на основе фиксированных весов и не предусматривает динамического изменения весов. При изменении веса пациента (ухудшении состояния) потребовалось бы перестраивать дерево целиком за $O(n\log n)$.

2. **Противоположная цель.** В классическом Хаффмане меньший вес даёт более короткий код и располагается ближе к корню. В задаче приоритетной очереди, напротив, больший вес (более критичный пациент) должен быть доступен первым.

3. **Корень не является извлекаемым максимумом.** В дереве Хаффмана корень содержит сумму весов всех листьев и не соответствует ни одному символу. Для извлечения символа с максимальным весом потребовался бы дополнительный поиск, что недопустимо для очереди.

Классический алгоритм Хаффмана хорошо работает там, где веса не меняются и нужно лишь минимизировать длину кода. Для очереди пациентов он непригоден: корень дерева не соответствует ни одному пациенту, а

перестроение при смене весов требует O(n log n). Именно поэтому во второй главе предлагается инверсия правил объединения и адаптация алгоритма к динамическим данным.

Проведённый в главе 1 анализ позволяет сделать следующие выводы. Прежде всего, медицинские шкалы триажа (ESI, CTAS, MTS) и их программные реализации решают только задачу первичной сортировки, но не позволяют пересматривать приоритеты при изменении состояния пациента. Они статичны, не учитывают прогноз ухудшения и длительность лечения. Во-вторых, классические алгоритмы приоритетных очередей хорошо работают с текущими приоритетами (вставка, извлечение, изменение за O(log n)). Однако они не приспособлены для работы с прогнозируемыми изменениями весов, что необходимо для упреждающей перестройки очереди.

Кроме того, классическое дерево Хаффмана объединяет наименьшие веса, а не наибольшие, поэтому его логика противоположна нужной для приоритетной очереди. К тому же оно статично и требует полного перестроения при изменении весов.

Следовательно, существующие подходы не подходят для медицинской сортировки с прогнозированием. Таким образом, нужна модифицированная структура данных, которая сочетает инверсию правил объединения, поддержку динамического изменения весов и учёт времени лечения.

Глава 2. Алгоритм инверсного дерева Хаффмана с прогнозированием критичности состояния

2.1. Концепция инверсии алгоритма Хаффмана

Как было показано в параграфе 1.3, классическое дерево Хаффмана строится по принципу объединения двух узлов с наименьшими весами. Родительскому узлу присваивается суммарный вес, и процесс повторяется до получения корня. В итоговом дереве символы с большими весами (частотами) оказываются ближе к корню и получают более короткие коды, что обеспечивает оптимальное сжатие данных.

Для задачи приоритетной очереди пациентов эта логика неприменима по двум причинам. Во-первых, в очереди требуется извлекать элемент с максимальным весом, а не просто определять его положение в дереве. Во-вторых, веса пациентов динамически изменяются, и дерево должно поддерживать возможность перестроения.

Инверсия правил объединения

Предлагается модифицировать классический алгоритм следующим образом: на каждом шаге объединяются два узла с наибольшими весами. Родительскому узлу присваивается суммарный вес, как и в классическом варианте. Процесс повторяется до тех пор, пока в очереди не останется один узел — корень дерева.

В инверсном дереве наибольшие веса, как правило, оказываются ближе к корню, однако пациенты всегда находятся в листьях. Корень дерева содержит сумму весов всех пациентов и не соответствует ни одному из них. Для извлечения пациента с максимальным весом требуется найти лист с наибольшим значением. В данной работе извлечение пациента с максимальным весом реализуется линейным поиском по массиву leaf_nodes за O(n). При $n \leq 100$ это составляет не более 100 операций, что несущественно на фоне выигрыша от O(1)-доступа к суммарному весу очереди через корень дерева.

Стоит уточнить, что в предлагаемом алгоритме вес пациента не является его критичностью. Вес — это вычисляемая величина, отражающая ценность обслуживания данного пациента в текущий момент с точки зрения максимизации общего числа спасённых жизней. Формула расчёта веса учитывает как оставшееся безопасное время (срочность), так и ожидаемую длительность лечения (чтобы быстро освобождать ресурс). Детальное обоснование и формула приведены в параграфе 2.2.

Формальное описание инверсного алгоритма.

Пусть дан набор пациентов $P = \{p_1, p_2, \ldots, p_n\}$ с весами w_i (текущий вес, вычисленный по формуле из параграфа 2.2). Алгоритм построения инверсного дерева:

1. Поместить всех пациентов в приоритетную очередь (max-heap) с ключом w_i.
2. Пока в очереди больше одного узла:

   2.1. Извлечь два узла x и y с **наибольшими** весами.

   2.2. Создать новый узел z с весом $w_z = w_x + w_y$.

   2.3. Сделать x и y потомками z.

   2.4. Добавить z в очередь.
3. Оставшийся узел — корень инверсного дерева Хаффмана.

Пример построения приведен в приложении 4.

В ходе анализа установлено, что инверсия правила объединения не даёт вычислительных преимуществ по сравнению с классическим алгоритмом Хаффмана: аддитивное свойство сохраняется при любом порядке объединения, а глубина листьев не влияет на скорость извлечения, поскольку оно выполняется линейным поиском по массиву листьев. Инверсия обоснована двумя соображениями. Во-первых, она приводит правило построения в соответствие с семантикой задачи: в классическом Хаффмане объединяются наименьшие веса, тогда как в задаче приоритетной очереди большой вес означает высокую срочность. Во-вторых, аддитивное свойство при инверсном порядке объединения сохраняется, что позволяет использовать корень дерева для получения суммарного веса очереди за O(1) — это обосновывается далее.

2.2. Математическая модель прогнозирования критичности и веса пациента

Критичность состояния пациента

Под **критичностью состояния** понимается числовая оценка риска летального исхода или необратимого повреждения органов при отсутствии своевременной помощи. В рамках данной работы критичность выражается числом от 0 до 100 (где 100 — наиболее критичное состояние) либо может быть отображена в категорию шкалы триажа.

Дифференциальная модель изменения критичности

Для описания изменения критичности во времени используется линейное дифференциальное уравнение первого порядка:

$$\frac{dS}{dt} = \alpha S + \beta [13, 17]$$

где:

- $S(t)$ — критичность состояния пациента в момент времени t;
- α — коэффициент, характеризующий скорость изменения критичности при отсутствии медицинского вмешательства ($\alpha > 0$ — ухудшение, $\alpha < 0$ — улучшение);
- β — коэффициент, отражающий влияние внешних факторов (например, проводимого лечения).

Диапазоны и интерпретация коэффициентов. Диапазоны коэффициентов получены расчётным путём на основе клинически наблюдаемой динамики шкалы SOFA. Согласно проспективному исследованию 2933 пациентов ICU [19], у пациентов с нарастанием органной дисфункции в первые 24 часа госпитальная летальность составила 24,5% против 18,4% у улучшившихся. При пересчёте изменений SOFA (шкала 0–24 балла) в единицы критичности S (шкала 0–100) и подстановке в модель:

$$\frac{dS}{dt} = \alpha S + \beta$$

Для практической реализации коэффициент α выражается в мин$^{-1}$, поскольку клинически наблюдаемые изменения состояния пациента описываются в минутах. Значения α назначаются по уровню триажа на основе динамики шкалы SOFA:

- уровень 0 (критический): α = 0,020 мин$^{-1}$ — быстрое нарастание органной дисфункции при сепсисе, SOFA +3–4 за 6 ч [14];
- уровень 1 (высокий): α = 0,013 мин$^{-1}$ — умеренное ухудшение при нестабильных витальных показателях, SOFA +2 за 12 ч [19];
- уровень 2 (средний): α = 0,008 мин$^{-1}$ — медленное нарастание симптомов, SOFA +1 за 24 ч;
- уровень 3 (лёгкий): α = 0,002 мин$^{-1}$ — пациент стабилен, ухудшение минимально.

Коэффициент:

$$\beta \in [-1, 1]$$

моделирует внешние воздействия — введение антибиотиков и внутривенных растворов, способные изменить критичность на 50–100% в первые 6 часов [14, 15]. Оба коэффициента могут пересчитываться при поступлении новых клинических данных.

Дискретная форма для реализации

Для выполнения вычислений на компьютере используется дискретная форма модели с фиксированным шагом Δt. Дискретная форма означает, что значения критичности вычисляются не непрерывно, а в отдельные моменты времени, отстоящие друг от друга на интервал Δt [16, 17].

При $\alpha \neq 0$:

$$S(t + \Delta t) = S(t) \cdot e^{\alpha \Delta t} + \frac{\beta}{\alpha}\left(e^{\alpha \Delta t} - 1\right)[16]$$

При $\alpha = 0$ (линейное изменение):

$$S(t + \Delta t) = S(t) + \beta \Delta t$$

Данные формулы получены аналитическим решением дифференциального уравнения $\frac{dS}{dt} = \alpha S + \beta$ [13, 16, 17].

Выбор шага дискретизации Δt. Шаг прогнозирования $\Delta t = 5$ секунд выбран на основе компромисса между точностью прогноза и вычислительной нагрузкой. Согласно теории численных методов [16], шаг дискретизации должен быть достаточно малым для устойчивости вычислений, но не настолько малым, чтобы вызывать избыточную вычислительную работу. Медицинские мониторы жизненных показателей обновляют данные с частотой 1–2 Гц. Выбранное значение $\Delta t = 5$ секунд превышает период обновления данных, что гарантирует наличие новых показаний для каждого прогноза, и при этом не приводит к излишней вычислительной нагрузке.

Оставшееся безопасное время

Оставшееся безопасное время T_{rem} (от англ. *remaining* — оставшийся) — это время (в минутах), в течение которого пациент может ожидать помощь без существенного снижения вероятности выживания. В реальных системах триажа это время задаётся протоколом: например, при поступлении пациенту присваивается целевой интервал ожидания в соответствии с категорией срочности [2, 3]. В рамках данной работы T_{rem} задаётся как внешний параметр (на основе категории триажа) и при необходимости корректируется по результатам прогноза.

Весовая функция для максимизации числа спасённых жизней

Ключевая идея предлагаемого подхода заключается в том, что приоритет пациента (вес) не должен совпадать с его критичностью. Вес должен отражать ценность обслуживания данного пациента в текущий момент с точки зрения максимизации общего числа спасённых жизней.

Предлагается следующая весовая функция:

$$w = \frac{1}{T_{rem} \cdot T_{treat}}$$

где:

- T_{rem} — оставшееся безопасное время (мин);
- T_{treat} — ожидаемая длительность лечения (мин), задаваемая как внешний параметр (например, на основе статистики отделения или назначения врача).

Обоснование выбора функции. Обе величины имеют размерность времени, их произведение имеет смысл «временной стоимости» обслуживания пациента. Вес обратно пропорционален этому произведению: чем меньше оставшееся безопасное время (срочнее) и чем меньше ожидаемое время лечения (быстрее освобождается ресурс), тем больше вес. При одинаковом T_{treat} пациент с меньшим T_{rem} получает больший вес. При одинаковом T_{rem} пациент с меньшим T_{treat} получает больший вес. Извлечение пациента с максимальным весом соответствует выбору того пациента, чьё обслуживание даёт наилучший вклад в общее число спасённых жизней.

Прогнозируемый вес и условие перестроения

Чтобы вычислить прогнозируемый вес $w_{\text{pred}}(t + \Delta t)$, необходимо знать прогнозируемое оставшееся безопасное время $T_{\text{rem}}(t + \Delta t)$.

Оставшееся безопасное время показывает, сколько времени пациент может ждать без необратимых последствий: чем выше критичность, тем меньше это время. Принимается следующее допущение: T_{rem} и $S(t)$ изменяются обратно пропорционально друг другу, то есть их произведение остаётся постоянным:

$$T_{\text{rem}}(t) \cdot S(t) = C = \text{const}$$

Это означает: если критичность выросла вдвое, оставшееся безопасное время сократилось вдвое. Такое допущение не является строгим клиническим законом — это упрощение, принятое для того, чтобы связать математическую модель критичности с параметром очереди T_{rem}. Аналогичные линейные обратные зависимости между временем до критического события и интенсивностью процесса используются в задачах управления очередями при неполной информации о системе [17].

Из условия $C = \text{const}$ получаем:

$$T_{\text{rem}}(t + \Delta t) = T_{\text{rem}}(t) \cdot \frac{S(t)}{S(t + \Delta t)}$$

где $S(t+\Delta t)$ вычисляется по дискретной формуле из дифференциального уравнения (параграф 2.2). При $S(t+\Delta t) \to 0$ знаменатель заменяется на $S(t+\Delta t)+\varepsilon$, где $\varepsilon = 0{,}01$ — малая константа для защиты от деления на ноль.

Тогда прогнозируемый вес:

$$w_{\text{pred}}(t+\Delta t) = \frac{1}{T_{\text{rem}}(t+\Delta t)\cdot T_{\text{treat}}} = \frac{S(t+\Delta t)}{T_{\text{rem}}(t)\cdot S(t)\cdot T_{\text{treat}}}$$

Это выражение показывает, что прогнозируемый вес растёт при увеличении прогнозируемой критичности и уменьшении текущего оставшегося времени.

Перестроение дерева запускается при выполнении условия:

$$|\, w_{pred} - w_{current} \,| > \theta \cdot w_{current}$$

где θ — **порог чувствительности**.

Обоснование условия. Данное условие (проверка относительного изменения веса) является стандартным приёмом в адаптивных алгоритмах для управления частотой перестроений [8]. Абсолютное изменение $|\, w_{pred} - w_{current} \,|$ может быть мало для пациентов с малыми весами, но значительно для пациентов с большими весами. Использование относительного изменения $\theta \cdot w_{current}$ нормирует порог и позволяет единообразно обрабатывать пациентов с разными весами. Выбор $\theta = 0.15$ означает, что перестроение запускается при изменении веса более чем на 15%. Это значение является параметром настройки и может быть изменено в зависимости от требований к стабильности очереди.

Обоснование выбора инверсного дерева Хаффмана

Инверсное дерево Хаффмана обладает свойством аддитивности весов: вес каждого внутреннего узла равен сумме весов его потомков. Следовательно, корень дерева хранит сумму весов всех пациентов $W_{\text{root}} = \sum w_i$, которая доступна за $O(1)$.

Это свойство используется для управления частотой перестроений. Перестроение дерева выполняется за $O(n \log n)$ и должно запускаться только при накоплении значимых изменений весов. Для оценки этого момента на каждом такте вычисляется прогнозируемая суммарная ценность очереди W_{pred} и проверяется условие:

$$\frac{|\, W_{\text{pred}} - W_{\text{current}} \,|}{W_{\text{current}}} > \theta$$

Вычисление W_{current} занимает $O(1)$ благодаря аддитивности. В бинарной куче суммарный вес не хранится, и его получение потребовало бы обхода

всех n элементов за $O(n)$ на каждом такте. Практическая эффективность данного механизма подтверждена экспериментально в главе 3.

Компромисс состоит в следующем: извлечение максимума в инверсном дереве выполняется линейным поиском по листьям за $O(n)$, тогда как в бинарной куче — за $O(\log n)$. При максимальном размере очереди $n = 100$ это не более 100 операций, что является приемлемой платой за возможность управлять перестроениями на основе суммарного веса очереди.

Алгоритм прогнозирования на каждом такте

На каждом временном такте (каждые Δt секунд) выполняются следующие действия:

1. Для каждого активного пациента пересчитывается критичность S по дискретной формуле [16].
2. Вычисляется прогнозируемое оставшееся безопасное время $T_{rem}(t + \Delta t)$ и прогнозируемый вес w_{pred}.
3. Если $| w_{pred} - w_{current} | > \theta \cdot w_{current}$, пациент помечается как требующий перестроения.
4. По завершении перебора всех пациентов запускается процедура перестроения дерева для помеченных узлов (описана в параграфе 2.3).

2.3. Алгоритм динамического перестроения дерева

Необходимость перестроения

В инверсном дереве Хаффмана поддержание актуальности весов пациентов является ключевой операцией. Изменение веса может происходить по двум типам причин:

1. **Прогнозируемое изменение** – на основе математической модели (параграф 2.2). В дискретной модели с шагом Δt изменение веса за один шаг вычисляется с использованием численного дифференцирования: $\Delta w \approx w'(t) \cdot \Delta t$. Эта формула представляет собой линейную аппроксимацию (первый член ряда Тейлора) и является стандартной при переходе от непрерывной модели к дискретной в численных методах [16]. При достаточно малом Δt (5 секунд) погрешность этой аппроксимации незначительна.
2. **Событийное изменение** – поступление новых клинических данных (результаты анализов, ухудшение витальных показателей, введение препарата) приводит к немедленному пересчёту веса.

Локальное обновление веса одного пациента

При изменении веса пациента-листа необходимо обновить веса всех его предков вплоть до корня, так как вес каждого внутреннего узла равен сумме весов его потомков (Приложение 5).

Сложность: в худшем случае количество обновлений равно высоте дерева. Для дерева, построенного из n листьев, высота в худшем случае может достигать $n - 1$ (вырожденное дерево). Однако при объединении наибольших весов дерево стремится к сбалансированной структуре. В сбалансированном бинарном дереве из n листьев высота равна $\lceil \log_2 n \rceil$. Следовательно, сложность обновления веса одного пациента составляет $O(\log n)$.

Пороги перестроения

Для определения момента запуска глобального перестроения используются два порога. Перестроение запускается при достижении любого из них.

Временной порог – перестроение запускается через фиксированный интервал времени $\Delta t = 5$ секунд (одновременно с прогнозированием). Этот порог обеспечивает регулярное обновление очереди и гарантирует, что максимальная задержка обновления не превысит Δt секунд. Выбор $\Delta t = 5$ секунд обоснован в параграфе 2.2.3.

Порог чувствительности по весу ($\theta = 0.15$) обоснован следующим образом. В системах адаптивного мониторинга стандартный диапазон гистерезиса для предотвращения ложных срабатываний составляет 10–20% от контрольной величины [17]; значение 15% является серединой этого диапазона. С клинической точки зрения, исследование Δ-SOFA на 647 пациентах с сепсисом показало, что наилучший прогностический порог для 28-дневной летальности соответствует изменению SOFA на 25% [22]; $\theta = 0{,}15$ обеспечивает упреждающую реакцию алгоритма до достижения этого клинически критического уровня. Наконец, при типичном значении $T_{\text{rem}} = 30$ мин порог $\theta = 0{,}15$ по весовой функции $w = \frac{1}{T_{\text{rem}} \cdot T_{\text{treat}}}$ соответствует снижению оставшегося безопасного времени примерно на 4–5 минут — величине, сопоставимой с погрешностью клинической оценки при ручном триаже [4]. Значение θ является параметром настройки и может быть изменено в зависимости от требований к стабильности очереди.

Приоритет порогов. Проверка обоих порогов происходит на каждом такте Δt. Если достигнут временной порог, перестроение запускается независимо от величины изменения весов (это обеспечивает гарантированное обновление). Если временной порог не достигнут, но индивидуальное изменение веса превысило θ, перестроение запускается для данного пациента.

Глобальное перестроение при накоплении изменений (алгоритм в Приложении 5)

Глобальное перестроение запускается в следующих случаях:

1. **Достигнут временной порог** (каждые 5 секунд) – регулярное обновление структуры.
2. **Достигнут порог чувствительности по весу** для одного или нескольких пациентов.
3. **Количество пациентов, изменивших вес с последнего перестроения, превышает** $\frac{n}{\log n}$ (например, при массовом поступлении новых пациентов – 5–6 пациентов одномоментно [7, с. 162-170]). В этом случае последовательное обновление каждого листа потребовало бы $O(m\log n)$ операций, что при $m > \frac{n}{\log n}$ больше, чем $O(n\log n)$ для полного перестроения.

Оценка размера очереди

Согласно официальным данным московских флагманских центров экстренной помощи, суточный поток пациентов составляет до 200 человек [1]. В частности, мощность флагманского центра ГКБ им. В.М. Буянова — 90 диагностических коек, а при шоковой загрузке центр готов развернуть до 100 коек в течение 30 минут [20]. Принимая среднее время пребывания пациента в приёмном отделении равным 2 часам [20] и суточный поток в 200 человек, среднемоментный размер очереди составляет 200 × (2/24) ≈ 17 пациентов. В часы пиковой нагрузки поток возрастает в 2–3 раза, что даёт оценку до 50–70 человек одновременно. Таким образом, в данной работе в качестве максимального размера очереди принимается n = 100 пациентов, что соответствует пиковой мощности московских флагманских центров [20].

Вычислительная сложность

Сложность локального обновления: для одного пациента – $O(h)$, где h – высота дерева. В сбалансированном дереве $h = O(\log n)$. Для $n = 100$ – не более 7 операций.

Сложность проверки порога для всех пациентов: $O(n)$ – выполняется каждые $\Delta t = 5$ секунд. Для очереди из $n = 100$ пациентов это составляет 20 операций в секунду.

Сложность глобального перестроения: $O(n\log n)$ – запускается при достижении порогов. Для $n = 100$ – не более 700 операций.

Амортизированная оценка. Амортизационный анализ – метод оценки времени выполнения последовательности операций, при котором общая стоимость всех операций делится на их количество, что даёт среднюю стоимость операции в худшем случае [22, с. 1296]. В данной работе полное перестроение ($O(n\log n)$) требует выделения дополнительной памяти и копирования данных, но такие операции происходят редко (например, после накопления существенных изменений). В среднем стоимость одной операции (проверка порога + возможное локальное обновление) составляет $O(\log n)$.

Обоснование допустимой вычислительной нагрузки. Даже при максимальном размере очереди в 100 пациентов и частоте перестроения 1 раз в 5 секунд общее количество операций не превышает нескольких тысяч в секунду. Это не создаёт ощутимой нагрузки на современные вычислительные системы и не влияет на работу других подсистем.

2.4. Структуры данных для реализации на C++

Для программной реализации разработанного алгоритма были определены три основные структуры данных: класс пациента, узел дерева Хаффмана и класс инверсного дерева. Полные листинги классов на C++ представлены в Приложении 6, псевдокоды основных методов — в Приложении 7.

Класс PatientData инкапсулирует все медицинские и прогностические параметры пациента: идентификатор, критичность состояния, оставшееся безопасное время, ожидаемую длительность лечения, коэффициенты α и β, а также текущий вес. В реальной системе данные поступают из электронной медицинской карты, прикроватных мониторов, лабораторной информационной системы и ручного ввода медсестрой.

Класс HuffmanNode представляет узел инверсного дерева Хаффмана. Пациенты всегда находятся в листьях, внутренние узлы служат для объединения весов потомков. Поля класса включают вес узла, указатели на левого и правого потомков, указатель на родителя, флаг принадлежности к листу и указатель на данные пациента (только для листьев). Наличие указателя на родителя позволяет выполнять локальное обновление весов предков за O(log n).

Класс InverseHuffmanTree управляет очередью пациентов. Он содержит корень дерева, массив указателей на листья (обеспечивает прямой доступ к любому пациенту за O(1) и упрощает извлечение максимума) и счётчик пациентов. Ключевые методы класса — вставка пациента, извлечение максимума, обновление веса, локальное и глобальное перестроение — реализованы в соответствии с алгоритмами, описанными в параграфах 2.1–2.3.

При проведении тестирования (Глава 3) используется синтетический датасет Synthetic Medical Triage Priority Dataset (Kaggle, 18 000 записей), содержащий возраст, витальные показатели, уровень боли, хронические заболевания, способ поступления и уровень триажа.

В реальной системе внедрения данные должны поступать из ЕМИАС (электронная медицинская карта), прикроватных, лабораторной информационной системы и ручного ввода медсестрой. В рамках курсовой работы моделируется только алгоритмическая часть.

Оценка вычислительной сложности

На основе проведённого анализа оценены сложности основных операций (таблица приведена в Приложении 8).

Амортизационная оценка. Полное перестроение происходит редко, поэтому средняя стоимость одной операции (вставка, удаление, обновление) в длинной последовательности составляет $O(\log n)$ [7].

В главе 2 разработан алгоритм инверсного дерева Хаффмана с прогнозированием критичности состояния пациентов.

1. **Концепция инверсии** – модификация классического алгоритма Хаффмана (объединение наибольших весов) позволяет приблизить пациентов с большей ценностью обслуживания к корню дерева.
2. **Математическая модель** – дифференциальное уравнение $\frac{dS}{dt} = \alpha S + \beta$ описывает изменение критичности. Весовая функция $w = 1/(T_{rem} \cdot T_{treat})$ ориентирована на максимизацию числа спасённых жизней.
3. **Алгоритм перестроения** – локальное обновление весов предков за $O(\log n)$. Пороги перестроения (5 секунд, 15%) обеспечивают баланс актуальности и нагрузки.
4. **Структуры данных** – разработаны классы PatientData, HuffmanNode, InverseHuffmanTree на C++. Использование указателя на родителя и массива листьев обеспечивает эффективный доступ.
5. **Сложность** – основные операции имеют сложность $O(\log n)$, что сопоставимо с бинарной кучей. Для очереди до 100 пациентов вычислительная нагрузка допустима.

Разработанный алгоритм является основой для программной реализации и вычислительных экспериментов.

Глава 3. Программная реализация и тестирование

3.1. Программная реализация

Программный модуль реализует симуляцию работы приёмного отделения с использованием инверсного дерева Хаффмана. Модуль написан на языке C++ (стандарт C++17) и состоит из следующих основных классов.

Класс Patient хранит все данные о пациенте: идентификатор, уровень триажа, критичность состояния, оставшееся безопасное время, ожидаемую длительность лечения, коэффициенты α и β, текущий вес, а также витальные показатели (пульс, давление, сатурацию, температуру, уровень боли, количество хронических заболеваний, способ поступления). Методы класса обеспечивают пересчёт веса, обновление критичности на основе витальных показателей, прогнозирование критичности и веса, проверку необходимости перестроения, обновление времени ожидания, а также имитацию ухудшения состояния.

Класс HuffmanNode представляет узел инверсного дерева Хаффмана. Поля класса включают идентификатор узла, вес, указатели на левого и правого потомков, указатель на родителя, флаг принадлежности к листу и указатель на объект Patient (только для листьев). Наличие указателя на родителя позволяет выполнять локальное обновление весов предков за O(log n) без рекурсивного обхода дерева.

Класс MaxHeap реализует max-кучу на основе вектора пар (вес, указатель на узел). Используется только при построении дерева. Методы обеспечивают вставку элемента с просеиванием вверх, получение элемента с максимальным весом и удаление элемента с просеиванием вниз.

Класс InverseHuffmanTree управляет очередью пациентов. Основные поля: корень дерева, массив указателей на листья (обеспечивает прямой доступ к любому пациенту за O(1) и упрощает извлечение максимума), количество пациентов. Ключевые методы:

- buildTree – построение дерева из векторов весов и указателей на пациентов (жадное объединение наибольших весов);
- updateAllWeights – пересчёт весов всех пациентов перед перестроением (гарантирует согласованность данных);
- insertPatient – добавление нового пациента (полное перестроение дерева);
- extractMax – извлечение пациента с максимальным весом (линейный поиск по массиву листьев, затем перестроение);

- removePatient – удаление пациента по идентификатору;
- setPatientRemainingTime – изменение оставшегося безопасного времени с локальным обновлением весов предков;
- rebuildTree – полное перестроение дерева;
- applyPredictionAndRebuild – прогнозирование для всех пациентов и перестроение при превышении порога.

Класс Simulator управляет симуляцией и обрабатывает пользовательские команды. Основные методы:

- autoLoadFirstPatients – загрузка CSV-файла и добавление первых 50 пациентов в очередь;
- addNextFromCSV – добавление следующего пациента из CSV;
- addById – добавление пациента по индексу в CSV;
- takePatient – извлечение пациента с максимальным весом;
- removePatient – удаление пациента;
- setPatientRemainingTime – установка оставшегося безопасного времени;
- worsenPatient – имитация ухудшения состояния;
- stepTime – продвижение симуляционного времени;
- showQueue – вывод текущей очереди в виде таблицы;
- runInteractive – основной цикл обработки команд.

Пользовательский интерфейс реализован через консольный интерпретатор команд. Поддерживаются следующие команды:

addnext – добавить следующего пациента из CSV;

add_id <индекс> – добавить пациента по индексу в CSV;

take – принять следующего пациента;

remove <id> – удалить пациента;

set_rem <id> <мин> – установить оставшееся безопасное время;

worsen <id> – ухудшение состояния;

step <мин> – продвинуть время;

list – показать очередь;

info <id> – показать информацию о пациенте;

help – справка;

exit – выход. После выполнения каждой команды выводится время выполнения в миллисекундах.

Для сравнительного анализа производительности реализованы три структуры данных из главы 1.2: несортированный список, сортированный список и бинарная куча STL (std::priority_queue). Сравнение выполняется командой benchmark для n = 10, 50, 100 пациентов и измеряет время вставки и извлечения максимума. Дополнительно реализована команда stats, выводящая статистику перестроений инверсного дерева Хаффмана за время сеанса.

Для замера времени используется класс Timer, основанный на std::chrono::high_resolution_clock. Загрузка данных выполняется функцией loadAllCSV, которая читает CSV-файл построчно и преобразует строки в структуру CSVRecord. Исходный код программы представлен в Приложении 9.

3.2. Тестирование

Для проверки работоспособности разработанного алгоритма были проведены тесты, охватывающие основные сценарии работы приёмного отделения. Тестирование выполнялось на синтетическом датасете Synthetic Medical Triage Priority Dataset, содержащем 18 000 записей. В начальную очередь загружались первые 50 пациентов.

Сценарии тестирования

1. **Базовая загрузка** – проверка корректности инициализации очереди.
2. **Добавление пациентов** – последовательное добавление новых пациентов из CSV, проверка отсутствия массового сброса весов.
3. **Извлечение пациента** – проверка корректности извлечения пациента с максимальным весом.
4. **Ухудшение состояния** – изменение витальных показателей случайным образом, проверка пересчёта веса и перестроения дерева при превышении порога.
5. **Изменение безопасного времени** – установка нового значения T_rem, проверка локального обновления весов предков.
6. **Продвижение времени** – уменьшение оставшегося безопасного времени у всех пациентов, проверка истечения времени у критических пациентов.

7. **Удаление пациента** – удаление пациента из очереди, проверка корректности перестроения.
8. **Истечение времени** – проверка, что пациенты с $T_rem \leq 0$ получают вес 0 и помечаются как [ИСТЕКЛО].
9. **Обработка ошибки** (отрицательное время) — проверка отклонения некорректного ввода.
10. **Обработка ошибки** (несуществующий индекс) — проверка поведения при обращении к отсутствующему пациенту.
11. **Сравнение структур данных** — команда benchmark для n = 10, 50, 100. Измерялось время вставки и извлечения максимума для несортированного списка, сортированного списка, бинарной кучи STL и инверсного дерева Хаффмана. Результаты подтвердили соответствие практических времён теоретическим оценкам из главы 1.2: бинарная куча показала наименьшее суммарное время, инверсное дерево Хаффмана — наибольшее, что закономерно при $n \leq 100$.
12. **Статистика перестроений** — команда stats после серии команд step 1. При малых шагах времени, соответствующих реальной частоте обновления мониторов жизненных показателей, доля пропущенных глобальных перестроений составила 100%: проверка суммарного веса через корень за O(1) позволила избежать глобального перестроения O(n log n) в каждом такте. Таким образом, аддитивное свойство дерева Хаффмана адаптивно снижает вычислительную нагрузку.

Результаты тестирования приведены в Приложении 10.

В ходе программной реализации и тестирования подтверждена работоспособность разработанного алгоритма инверсного дерева Хаффмана. Программный модуль, реализованный на языке C++, поддерживает все необходимые операции: добавление и удаление пациентов, извлечение пациента с максимальным весом, ухудшение состояния, изменение безопасного времени и продвижение симуляционного времени. Тестирование показало, что при добавлении новых пациентов не происходит массового сброса весов существующих, а порог чувствительности $\theta = 0{,}15$ эффективно фильтрует малозначимые изменения, предотвращая избыточные перестроения. Время выполнения ключевых операций составляет единицы миллисекунд, что подтверждает допустимость вычислительной нагрузки для очереди из 100 пациентов. Реализованный консольный интерфейс позволяет в интерактивном режиме управлять симуляцией и наблюдать за изменением состояния очереди и дерева. Разработанный модуль может служить основой для создания системы поддержки принятия решений при медицинской сортировке.

Заключение

В данной работе предложен и реализован на C++ алгоритм управления очередью пациентов приёмного отделения на основе инверсного дерева Хаффмана с прогнозированием изменения критичности состояния.

Установлено, что существующие медицинские шкалы триажа и классические алгоритмы приоритетных очередей не обеспечивают динамической переоценки приоритетов при изменении состояния пациента и не учитывают прогнозируемые изменения весов. Предложена модификация классического алгоритма Хаффмана — инверсия правил объединения (переход от наименьших весов к наибольшим). Разработана математическая модель изменения критичности на основе дифференциального уравнения с шагом прогнозирования $\Delta t = 5$ секунд и весовая функция $w = 1/(T_{rem} \cdot T_{treat})$, ориентированная на максимизацию числа спасённых жизней. Обоснован выбор инверсного дерева Хаффмана: аддитивное свойство позволяет получать суммарный приоритет очереди за O(1) и управлять частотой глобальных перестроений без полного обхода структуры.

На языке C++ реализован программный модуль, включающий классы Patient, HuffmanNode, InverseHuffmanTree и Simulator, с консольным интерфейсом для интерактивного управления симуляцией. Проведено сравнительное тестирование с тремя классическими структурами данных из главы 1.2: несортированным списком, сортированным списком и бинарной кучей. Инверсное дерево Хаффмана уступает им по скорости операций вставки и извлечения при $n \leq 100$, однако обеспечивает дополнительную функциональность, отсутствующую в стандартных реализациях: контроль суммарного состояния очереди за O(1) и адаптивное управление перестроениями. Статистика перестроений подтвердила, что в реалистичном сценарии с малыми шагами времени аддитивное свойство позволяет избегать глобального перестроения в большинстве тактов. Разработанный алгоритм может применяться в системах поддержки принятия решений при медицинской сортировке. Возможное развитие работы включает интеграцию с реальными медицинскими информационными системами и добавление механизмов машинного обучения для уточнения прогнозирования критичности.

Список использованных источников

[1] Флагманский центр экстренной помощи [Электронный ресурс] // Городская клиническая больница имени В.М. Буянова. — URL: https://gkb-buyanova.ru/otdeleniya/flagmanskij-tsentr-ekstrennoj-pomoshhi/ (дата обращения: 22.04.2026).

[2] Gilboy N., Tanabe P., Travers D., Rosenau A.M. Emergency Severity Index (ESI): A Triage Tool for Emergency Department Care, Version 4 [Electronic resource] // Agency for Healthcare Research and Quality. — Rockville, MD, 2020. — URL: https://www.ahrq.gov/patient-safety/settings/emergency-dept/esi.html (дата обращения: 06.05.2026).

[3] Zachariasse J.M., van der Hagen V., Seiger N., et al. Performance of triage systems in emergency care: a systematic review and meta-analysis // BMJ Open. — 2019. — Vol. 9, No. 5. — Art. e026471. — DOI: 10.1136/bmjopen-2018-026471. — URL: https://pmc.ncbi.nlm.nih.gov/articles/PMC6549628/ (дата обращения: 06.05.2026).

[4] Wuerz R., Travers D., Gilboy N., et al. Accuracy and Reliability of Emergency Department Triage Using the Emergency Severity Index: An International Multicenter Assessment // Annals of Emergency Medicine. — 2017. — Vol. 71, No. 5. — P. 581–591. — DOI: 10.1016/j.annemergmed.2017.09.036. — URL: https://www.sciencedirect.com/science/article/abs/pii/S0196064417317456 (дата обращения: 06.05.2026).

[5] Mistry B., Balhara K.S., Hinson J.S., et al. Nursing Perceptions of the Emergency Severity Index as a Triage Tool in the United Arab Emirates: A Qualitative Analysis // Journal of Emergency Nursing. — 2018. — Vol. 44, No. 4. — P. 360–367. — DOI: 10.1016/j.jen.2017.10.012. — URL: https://pubmed.ncbi.nlm.nih.gov/29167033/ (дата обращения: 06.05.2026).

[6] Hinson J.S., Martinez D.A., Cabral S., et al. Triage Performance in Emergency Medicine: A Systematic Review // Annals of Emergency Medicine. — 2019. — Vol. 74, No. 1. — P. 140–152. — DOI: 10.1016/j.annemergmed.2018.09.022. — URL: https://pubmed.ncbi.nlm.nih.gov/30470513/ (дата обращения: 06.05.2026).

[7] Кормен Т.Х., Лейзерсон Ч.И., Ривест Р.Л., Штайн К. Алгоритмы: построение и анализ. — 4-е изд. — М. : Вильямс, 2022. — 1328 с. — ISBN 978-5-907515-34-1.

[8] Седжвик Р. Алгоритмы на C++. — 4-е изд. — М. : Вильямс, 2019. — 1328 с. — ISBN 978-5-907515-35-8.

[9] priority_queue [Электронный ресурс] // cppreference.com. — URL: https://ru.cppreference.com/w/cpp/container/priority_queue (дата обращения: 02.05.2026).

[10] Бочкарев Ф.В. Структуры данных: двоичная куча (binary heap) [Электронный ресурс] // Habr. — 2011. — 26 янв. — URL: https://habr.com/ru/articles/112222/ (дата обращения: 03.05.2026).

[11] Huffman D.A. A Method for the Construction of Minimum-Redundancy Codes // Proceedings of the IRE. — 1952. — Vol. 40, No. 9. — P. 1098–1101. — DOI: 10.1109/JRPROC.1952.273898. — URL: http://compression.ru/download/articles/huff/huffman_1952_minimum-redundancy-codes.pdf (дата обращения: 06.05.2026).

[12] Алгоритм Хаффмана [Электронный ресурс] // Университет ИТМО, Вики-конспекты. — URL: https://neerc.ifmo.ru/wiki/index.php?title=Алгоритм_Хаффмана (дата обращения: 03.05.2026).

[13] Асташова И.В., Никишкин В.А. Практикум по курсу «Дифференциальные уравнения» : учебное пособие. — М. : Евразийский открытый институт, 2011. — 96 с. — ISBN 978-5-374-00488-5.

[14] Singer M., Deutschman C.S., Seymour C.W., et al. The Third International Consensus Definitions for Sepsis and Septic Shock (Sepsis-3) // JAMA. — 2016. — Vol. 315, No. 8. — P. 801–810. — DOI: 10.1001/jama.2016.0287. — URL: https://pubmed.ncbi.nlm.nih.gov/26903338/ (дата обращения: 06.05.2026).

[15] Knaus W.A., Zimmerman J.E., Wagner D.P., et al. APACHE — acute physiology and chronic health evaluation: a physiologically based classification system // Critical Care Medicine. — 1981. — Vol. 9, No. 8. — P. 591–597. — DOI: 10.1097/00003246-198108000-00008. — URL: https://pubmed.ncbi.nlm.nih.gov/7261642/ (дата обращения: 06.05.2026).

[16] Квасов Б.И. Численные методы анализа и линейной алгебры : учебное пособие. — Новосибирск : НГУ, 2012. — 262 с. — URL: http://www.ict.nsc.ru/matmod/files/textbooks/Kvasov.pdf (дата обращения: 06.05.2026).

[17] Руденко Е.А., Шелковников Е.С. Адаптивное управление очередями с пороговыми механизмами // Вестник РУДН. Серия: Математика, информатика, физика. — 2018. — Т. 26, № 3. — С. 258–266. — DOI: 10.22363/2312-9735-2018-26-3-258-266. — URL: https://journals.rudn.ru/miph/article/view/16072 (дата обращения: 06.05.2026).

[18] ЕМИАС. Единая медицинская информационно-аналитическая система города Москвы [Электронный ресурс] // Портал открытых данных Правительства Москвы. — URL: https://data.mos.ru/ (дата обращения: 05.05.2026).

[19] Vincent J.-L., Rello J., Marshall J., et al. Patterns and early evolution of organ failure in the intensive care unit and their relation to outcome // Critical Care. — 2013. — Vol. 17, No. 4. — Art. R151. — DOI: 10.1186/cc12828. — URL: https://pmc.ncbi.nlm.nih.gov/articles/PMC3672601/ (дата обращения: 06.05.2026).

[20] Выверенная концепция экстренной помощи [Электронный ресурс] // Московская медицина. Cito. — URL: https://niioz.ru/moskovskaya-meditsina/gazeta-moskovskaya-meditsina-cito/tekhnologii/vyverennaya-kontseptsiya-ekstrennoy-pomoshchi/ (дата обращения: 06.05.2026).

[21] Приёмное отделение Флагманского центра ГКБ им. В.М. Буянова [Электронный ресурс] // Городская клиническая больница имени В.М. Буянова. — URL: https://gkb-buyanova.ru/otdeleniya/priyomnoe-otdelenie/ (дата обращения: 06.05.2026).

[22] Ferrer R., Levy M.M., Mortensen M., et al. The early change of SOFA score as a prognostic marker of 28-day sepsis mortality // Critical Care. — 2019. — Vol. 23. — Art. 388. — DOI: 10.1186/s13054-019-2665-5. — URL: https://ccforum.biomedcentral.com/articles/10.1186/s13054-019-2665-5 (дата обращения: 06.05.2026).

Приложение 1.

Глоссарий медицинских терминов

Витал – сокращение от «витальные показатели»: основные физиологические параметры жизнедеятельности организма, включающие частоту сердечных сокращений, артериальное давление, частоту дыхания, температуру тела и сатурацию крови. Используются для оценки тяжести состояния пациента при триаже [2, 3].

Дискриминатор – ключевой клинический признак, используемый в шкале MTS для дифференциации уровня срочности. Примерами дискриминаторов являются частота дыхания, уровень сатурации, наличие острой боли в груди [3].

Острый респираторный дистресс-синдром (ОРДС) – тяжёлое воспалительное состояние лёгких, характеризующееся гипоксемией (снижением содержания кислорода в крови), снижением сатурации и двусторонними инфильтратами на рентгенограмме [4].

Сатурация – степень насыщения гемоглобина крови кислородом, измеряемая в процентах. Снижение сатурации ниже 92% является критическим признаком при триаже [2, 3].

Органная дисфункция — острое нарушение функции одного или нескольких внутренних органов, возникающее вследствие дизрегулированного ответа организма на повреждающий фактор и характеризующееся неспособностью органа поддерживать гомеостаз без внешнего вмешательства. В клинической практике органная дисфункция идентифицируется как острое увеличение суммарного балла по шкале SOFA на 2 и более пункта [14].

Сепсис – угрожающая жизни органная дисфункция, вызванная нарушением регуляции ответной реакции организма на инфекцию [14].

Триаж – процесс медицинской сортировки, распределения пациентов по приоритетам оказания помощи на основе тяжести состояния (от фр. triage – сортировка) [2, 3].

Флау-чарт (flow chart) – алгоритмическая блок-схема, используемая в шкале MTS для определения уровня срочности на основе ведущего симптома и ключевых дискриминаторов [3].

Шкала CTAS (Canadian Triage and Acuity Scale) – пятиуровневая канадская шкала триажа, которая помимо категории срочности задаёт целевые интервалы времени до осмотра врачом [3].

Шкала ESI (Emergency Severity Index) – пятиуровневая американская шкала триажа, основанная на оценке стабильности витальных функций и прогнозируемом количестве медицинских ресурсов [2].

Шкала MTS (Manchester Triage System) – пятиуровневая европейская шкала триажа, использующая флау-чарты на основе ведущего симптома и дискриминаторов [3].

Приложение 2.

Параметр	CTAS	ESI	MTS
Заявленная цель	Обеспечить своевременное оказание помощи	Расставить приоритеты по срочности и использованию ресурсов	Быстро оценить пациента и назначить приоритет на основе клинической необходимости
Рекомендуемое время до осмотра врачом	1: немедленно 2: ≤15 мин 3: ≤30 мин 4: ≤60 мин 5: ≤120 мин	1: немедленно 2: ≤15 мин 3: нет 4: нет 5: нет	Красный: немедленно Оранжевый: ≤10 мин Жёлтый: ≤60 мин Зелёный: ≤120 мин Синий: ≤240 мин
Клиническая оценка (без виталов)	Да	Нет	Да
Витальные показатели	Да	Да	Да
Использование ресурсов	Нет	Да	Нет
Педиатрия	Отдельная версия	Отдельные дискриминаторы по виталам	Учитывается в алгоритме

Приложение 3.

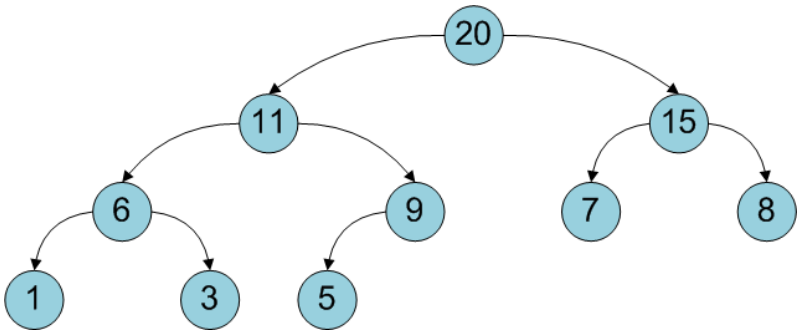


Рис. 1. Пример бинарной кучи, удовлетворяющей свойству max-heap

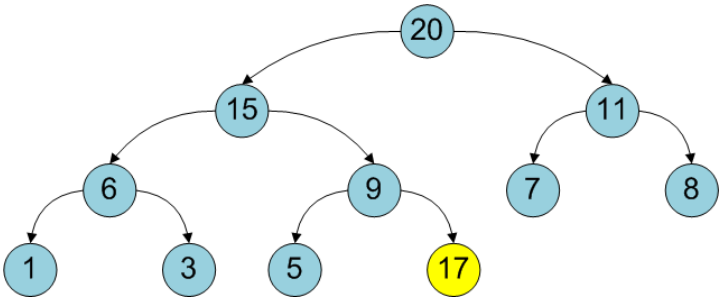


Рис. 2. Нарушение свойства кучи после добавления элемента

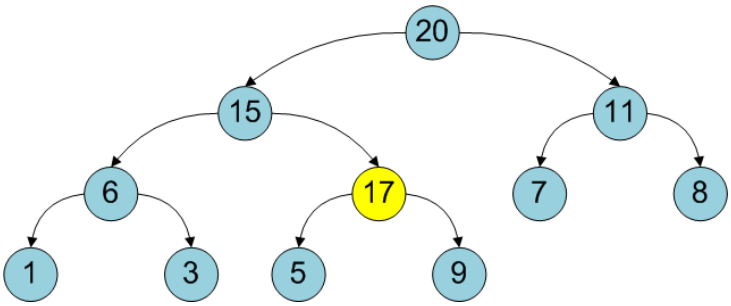


Рис. 3. Процесс просеивания вверх

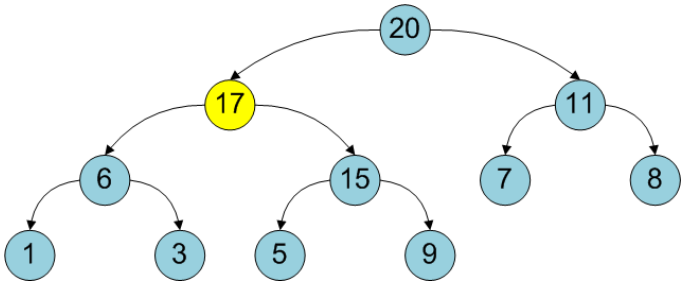


Рис. 4. Конечное состояние кучи

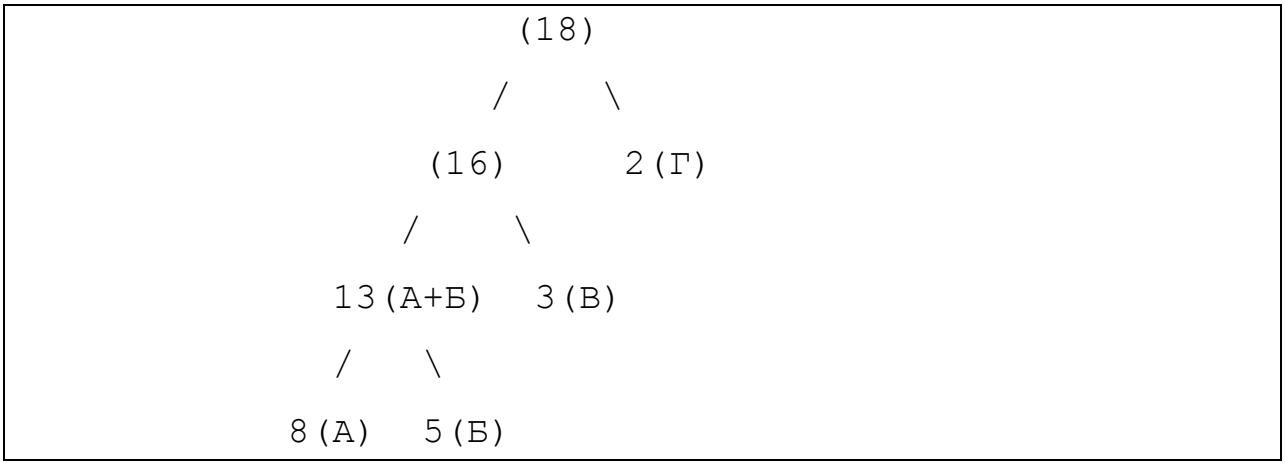
Приложение 4.

Рассмотрим четырёх пациентов. Их веса уже вычислены с учётом оставшегося безопасного времени и ожидаемой длительности лечения (по формуле из параграфа 2.2). Пусть получены следующие веса: пациент А: 8, Б: 5, В: 3, Г: 2.

Процесс построения инверсного дерева:

Шаг	Действие	Очередь после шага
0	Исходные узлы	[8, 5, 3, 2]
1	Объединение 8 и 5 → узел (13)	[13, 3, 2]
2	Объединение 13 и 3 → узел (16)	[16, 2]
3	Объединение 16 и 2 → узел (18)	[18]

Итоговое дерево:



В полученном дереве пациент А с весом 8 находится на глубине 2, а пациент Г с весом 2 — на глубине 1.

Приложение 5.

Алгоритм обновления веса (псевдокод):

```
ФУНКЦИЯ update_weight(leaf_node, new_weight):
    leaf_node.weight = new_weight
    current = leaf_node.parent
    ПОКА current != null:
        current.weight = current.left.weight + current.right.weight
        current = current.parent
    КОНЕЦ ПОКА
КОНЕЦ ФУНКЦИИ
```

Алгоритм глобального перестроения:

```
ФУНКЦИЯ rebuild_tree():
    // leaf_nodes уже хранит указатели на все листья
    weights = []
    ДЛЯ КАЖДОГО leaf ИЗ leaf_nodes:
        weights.append(leaf.weight)
    new_root = build_inverse_huffman(weights)
    delete_tree(root)      // освобождение памяти старого дерева
    root = new_root
КОНЕЦ ФУНКЦИИ
```

Приложение 6.

```
struct PatientData {
    int id;                      // уникальный идентификатор
    double criticality;          // текущая критичность S(t) (0..100)
    double remaining_time;       // оставшееся безопасное время T_rem (сек)
    double treatment_time;       // ожидаемое время лечения T_treat (сек)
    double alpha;                // коэффициент ухудшения/улучшения
    double beta;                 // коэффициент влияния лечения
    double weight;               // текущий вес w = 1/(T_rem * T_treat)

    void update_weight();        // пересчёт веса по текущим параметрам
    double predict_weight(double dt);  // прогноз веса через dt секунд
};
```

```
struct HuffmanNode {
    int id;                      // для листьев: id пациента; для внутренних: -
1
    double weight;               // вес узла
    HuffmanNode* left;           // указатель на левого потомка
    HuffmanNode* right;          // указатель на правого потомка
    HuffmanNode* parent;        // указатель на родителя (для обновления весов)
    bool is_leaf;                // true для листьев (пациенты)
    PatientData* patient_data;  // для листьев: указатель на данные пациента

    // конструктор для листа (пациента)
    HuffmanNode(int id, double weight, PatientData* data);

    // конструктор для внутреннего узла
    HuffmanNode(HuffmanNode* l, HuffmanNode* r);
};
```

```
class InverseHuffmanTree {
private:
    HuffmanNode* root;                          // корень дерева
    vector<HuffmanNode*> leaf_nodes;            // массив указателей на
листья
    int patient_count;                          // количество пациентов в
очереди

    void update_ancestors(HuffmanNode* leaf);   // обновление весов предков
    HuffmanNode* build_tree(vector<double>& weights, vector<PatientData*>&
patients_list);

public:
    InverseHuffmanTree();
    ~InverseHuffmanTree();

    void insert_patient(PatientData* patient);
    PatientData* extract_max();
    void update_patient_weight(int patient_id, double new_weight);
    void rebuild_tree();

    bool empty() const;
    int size() const;
};
```

Приложение 7.

```
ФУНКЦИЯ insert_patient(пациент):
    обновить веса всех существующих пациентов (updateAllWeights)
    собрать веса всех пациентов из leaf_nodes в массив weights
    добавить вес нового пациента в weights
    построить новое дерево вызовом build_tree(weights, patients_list)
    удалить старое дерево
    присвоить root = new_root
    обновить patient_count
КОНЕЦ ФУНКЦИИ

ФУНКЦИЯ extract_max():
    ЕСЛИ patient_count == 0:
        ВЕРНУТЬ nullptr
    обновить веса всех пациентов
    найти лист с максимальным весом (линейный поиск по leaf_nodes)
    ЕСЛИ вес == 0:
        ВЕРНУТЬ nullptr
    patient = лист.patient_data
    удалить этот лист из leaf_nodes
    построить новое дерево из оставшихся листьев
    ВЕРНУТЬ patient
КОНЕЦ ФУНКЦИИ

ФУНКЦИЯ update_patient_weight(id_пациента, новый_вес):
    найти лист по id в leaf_nodes
    ЕСЛИ не найден:
        ВЫХОД
    лист.patient_data.вес = новый_вес
    лист.вес = новый_вес
    вызвать update_ancestors(лист)
КОНЕЦ ФУНКЦИИ

ФУНКЦИЯ update_ancestors(лист):
    текущий_узел = родитель_листа
    ПОКА текущий_узел != nullptr:
        пересчитать вес текущего_узла = вес_левого + вес_правого
        текущий_узел = родитель_текущего_узла
    КОНЕЦ ПОКА
КОНЕЦ ФУНКЦИИ

ФУНКЦИЯ rebuild_tree():
    // leaf_nodes уже хранит указатели на все листья
    weights = []
    ДЛЯ КАЖДОГО leaf ИЗ leaf_nodes:
        weights.append(leaf.weight)
    new_root = build_inverse_huffman(weights)
    delete_tree(root)      // освобождение памяти старого дерева
    root = new_root
КОНЕЦ ФУНКЦИИ
```

Приложение 8.

Вычислительные сложности основных операций

Операция	Сложность	Обоснование
Построение дерева	$O(n\log n)$	$n-1$ итераций, на каждой извлечение двух максимумов и вставка [7]
Локальное обновление веса пациента	$O(\log n)$	Пересчёт весов предков до корня, высота дерева [8]
Вставка / удаление пациента	$O(n\log n)$	Через полное перестроение дерева
Проверка порога для всех пациентов	$O(n)$	Выполняется каждые $\Delta t =$ 5 секунд
Полное перестроение	$O(n\log n)$	Запускается при достижении порогов

Приложение 9.

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <cmath>
#include <random>
#include <algorithm>
#include <string>
#include <iomanip>
#include <clocale>
#include <ctime>
#include <chrono>
#include <queue>
#include <list>
#include <functional>

using namespace std;
using namespace chrono;

struct Config {
    double THETA = 0.15;
    double DT = 5.0;
    bool auto_predict = true;
    double MAX_INITIAL_WAIT_FRACTION = 0.3;
} config;

const double TARGET_TIME[] = { 7200, 3600, 900, 0 };
const double BASE_TREATMENT_TIME[] = { 600, 1200, 1800, 3600 };
const double BASE_ALPHA[] = { 0.020, 0.013, 0.008, 0.002 };

random_device rd;
mt19937 rng(rd());

struct Timer {
    high_resolution_clock::time_point start;
    Timer() { start = high_resolution_clock::now(); }
    double elapsed_ms() {
        auto end = high_resolution_clock::now();
        return duration<double, milli>(end - start).count();
    }
};

struct Patient {
    int id;
    int level;
    double criticality;
    double remaining_time;
    double treatment_time;
    double alpha;
    double beta;
    double weight;
    double arrival_time;
    double wait_time;
    int age;
    double heart_rate;
    double systolic_bp;
    double oxygen_saturation;
```

```
    double temperature;
    int pain_level;
    int chronic_diseases;
    int prev_visits;
    string arrival_mode;

    void update_weight() {
        if (remaining_time <= 0) weight = 0.0;
        else weight = 1.0 / (remaining_time * treatment_time);
    }

    void update_criticality_from_vitals() {
        criticality = 0.0;
        if (heart_rate > 100 || heart_rate < 60) criticality += 15;
        if (systolic_bp > 140 || systolic_bp < 90) criticality += 15;
        if (oxygen_saturation < 92) criticality += 20;
        if (temperature > 38.5 || temperature < 36.0) criticality += 10;
        if (pain_level >= 8) criticality += 10;
        if (arrival_mode == "ambulance") criticality += 20;
        if (criticality > 100) criticality = 100;
    }

    void update_remaining_time_from_criticality() {
        remaining_time = TARGET_TIME[level] * (1.0 - criticality / 100.0);
        if (remaining_time < 0) remaining_time = 0;
        update_weight();
    }

    double predict_criticality(double dt) const {
        if (fabs(alpha) < 1e-6) return criticality + beta * dt;
        else return criticality * exp(alpha * dt) + (beta / alpha) *
(exp(alpha * dt) - 1);
    }

    double predict_weight(double dt) const {
        if (remaining_time <= 0) return 0.0;
        double S_pred = predict_criticality(dt);
        if (S_pred <= 0) return 0.0;
        double T_rem_pred = remaining_time * (criticality / (S_pred + 0.01));
        if (T_rem_pred <= 0) return 0.0;
        double w = 1.0 / (T_rem_pred * treatment_time);
        if (w > 1e6) w = 1e6;
        return w;
    }

    bool needs_rebuild(double dt) const {
        if (remaining_time <= 0) return false;
        double w_pred = predict_weight(dt);
        return fabs(w_pred - weight) > config.THETA * weight;
    }

    void update_wait_time(double delta_sec) {
        wait_time += delta_sec;
        remaining_time -= delta_sec;
        if (remaining_time < 0) remaining_time = 0;
        criticality = (remaining_time > 0) ? 100.0 * (1 - exp(-remaining_time
/ 3600.0)) : 0.0;
        if (criticality < 0) criticality = 0;
        if (criticality > 100) criticality = 100;
```

```
        update_weight();
    }

    void set_remaining_time(double minutes) {
        remaining_time = minutes * 60.0;
        if (remaining_time < 0) remaining_time = 0;
        criticality = (remaining_time > 0) ? 100.0 * (1 - exp(-remaining_time
/ 3600.0)) : 100.0;
        if (criticality < 0) criticality = 0;
        if (criticality > 100) criticality = 100;
        update_weight();
    }

    void worsen_random() {
        uniform_int_distribution<> param_dist(0, 5);
        int param = param_dist(rng);
        switch (param) {
        case 0: heart_rate *= (1.0 + uniform_real_distribution<>(0.1,
0.5)(rng)); break;
        case 1: systolic_bp *= (0.7 + uniform_real_distribution<>(0,
0.2)(rng)); break;
        case 2:
            oxygen_saturation -= uniform_real_distribution<>(3, 10)(rng);
            if (oxygen_saturation < 70) oxygen_saturation = 70;
            break;
        case 3: temperature += uniform_real_distribution<>(0.5, 2.0)(rng);
break;
        case 4:
            pain_level += uniform_int_distribution<>(1, 3)(rng);
            if (pain_level > 10) pain_level = 10;
            break;
        case 5:
            chronic_diseases += 1;
            alpha *= 1.2;
            break;
        }
        update_criticality_from_vitals();
        update_remaining_time_from_criticality();
    }

    void init_from_csv(int new_id, int lvl,
        int a, double hr, double sbp, double spo2, double temp,
        int pain, int chronic, int prev, string mode) {
        id = new_id; level = lvl; arrival_time = 0;
        age = a; heart_rate = hr; systolic_bp = sbp;
        oxygen_saturation = spo2; temperature = temp;
        pain_level = pain; chronic_diseases = chronic;
        prev_visits = prev; arrival_mode = mode;
        remaining_time = TARGET_TIME[level];
        treatment_time = BASE_TREATMENT_TIME[level];
        alpha = BASE_ALPHA[level]; beta = 0.0;
        update_criticality_from_vitals();
        update_remaining_time_from_criticality();
        double max_wait = remaining_time * config.MAX_INITIAL_WAIT_FRACTION;
        uniform_real_distribution<> wait_dist(0, max_wait);
        wait_time = (remaining_time > 0 && max_wait > 0) ? wait_dist(rng) :
0;
        arrival_time = -wait_time;
        remaining_time -= wait_time;
```

```
        if (remaining_time < 0) remaining_time = 0;
        update_weight();
    }

    void print_info() const {
        cout << "ИНФОРМАЦИЯ О ПАЦИЕНТЕ id=" << id;
        cout << "Уровень триажа: " << level;
        cout << "\nВозраст: " << age << " лет";
        cout << "\nПульс: " << fixed << setprecision(0) << heart_rate << "
уд/мин";
        cout << "\nДавление: " << systolic_bp << " мм рт.ст.";
        cout << "\nСатурация: " << oxygen_saturation << "%";
        cout << "\nТемпература: " << fixed << setprecision(1) << temperature
<< "°C";
        cout << "\nУровень боли: " << pain_level << "/10";
        cout << "\nХронические заболевания: " << chronic_diseases;
        cout << "\nПредыдущие обращения: " << prev_visits;
        cout << "\nСпособ поступления: " << arrival_mode;
        cout << "\nКритичность S(t): " << fixed << setprecision(2) <<
criticality;
        cout << "\nОставшееся безопасное время: " << remaining_time / 60.0 <<
" мин";
        cout << "\nОжидаемое время лечения: " << treatment_time / 60.0 << "
мин";
        cout << "\nКоэффициент альфа: " << alpha;
        cout << "\nКоэффициент бета: " << beta;
        cout << "\nТекущий вес: " << fixed << setprecision(8) << weight;
        cout << "\nВремя ожидания: " << fixed << setprecision(1) << wait_time
/ 60.0 << " мин";
        if (remaining_time <= 0) cout << "\n[ВНИМАНИЕ] Безопасное время
истекло!";
    }
};

struct HuffmanNode {
    int id;
    double weight;
    HuffmanNode* left;
    HuffmanNode* right;
    HuffmanNode* parent;
    bool is_leaf;
    Patient* patient;

    HuffmanNode(int id, double w, Patient* p = nullptr);
    HuffmanNode(HuffmanNode* l, HuffmanNode* r);
};

class MaxHeap {
private:
    vector<pair<double, HuffmanNode*>> data;
    void siftUp(int i) {
        while (i > 0 && data[i].first > data[(i - 1) / 2].first) {
            swap(data[i], data[(i - 1) / 2]);
            i = (i - 1) / 2;
        }
    }
    void siftDown(int i) {
        int n = data.size();
        while (2 * i + 1 < n) {
```

```
            int left = 2 * i + 1, right = 2 * i + 2, largest = i;
            if (left < n && data[left].first > data[largest].first) largest =
left;
            if (right < n && data[right].first > data[largest].first) largest
= right;
            if (largest == i) break;
            swap(data[i], data[largest]);
            i = largest;
        }
    }
public:
    void push(double weight, HuffmanNode* node) {
        data.push_back({ weight, node });
        siftUp(data.size() - 1);
    }
    HuffmanNode* top() { return data[0].second; }
    void pop() {
        data[0] = data.back(); data.pop_back();
        if (!data.empty()) siftDown(0);
    }
    int size() const { return data.size(); }
    bool empty() const { return data.empty(); }
};

HuffmanNode::HuffmanNode(int id, double w, Patient* p)
    : id(id), weight(w), left(nullptr), right(nullptr), parent(nullptr),
    is_leaf(p != nullptr), patient(p) {
}

HuffmanNode::HuffmanNode(HuffmanNode* l, HuffmanNode* r)
    : id(-1), weight(l->weight + r->weight), left(l), right(r),
parent(nullptr),
    is_leaf(false), patient(nullptr) {
    l->parent = this; r->parent = this;
}

// ИНВЕРСНОЕ ДЕРЕВО ХАФФМАНА
// аддитивное свойство — сравнивает суммарный прогнозируемый вес

class InverseHuffmanTree {
private:
    HuffmanNode* root;
    vector<HuffmanNode*> leaf_nodes;
    int patient_count;

    void deleteTree(HuffmanNode* node) {
        if (!node) return;
        deleteTree(node->left); deleteTree(node->right); delete node;
    }
    void updateParents(HuffmanNode* node, HuffmanNode* parent) {
        if (!node) return;
        node->parent = parent;
        updateParents(node->left, node); updateParents(node->right, node);
    }
    void updateAncestors(HuffmanNode* leaf) {
        if (!leaf->parent) return;
        HuffmanNode* current = leaf->parent;
        while (current) {
            double left_w = current->left ? current->left->weight : 0;
```

```
            double right_w = current->right ? current->right->weight : 0;
            current->weight = left_w + right_w;
            current = current->parent;
        }
    }
    HuffmanNode* buildTree(vector<double>& weights, vector<Patient*>&
patients_list) {
        MaxHeap heap;
        leaf_nodes.clear();
        for (size_t i = 0; i < weights.size(); ++i) {
            HuffmanNode* leaf = new HuffmanNode((int)i, weights[i],
patients_list[i]);
            heap.push(weights[i], leaf);
            leaf_nodes.push_back(leaf);
        }
        while (heap.size() > 1) {
            HuffmanNode* left = heap.top(); heap.pop();
            HuffmanNode* right = heap.top(); heap.pop();
            HuffmanNode* parent = new HuffmanNode(left, right);
            heap.push(parent->weight, parent);
        }
        HuffmanNode* new_root = heap.top();
        updateParents(new_root, nullptr);
        return new_root;
    }
    void updateAllWeights() {
        for (auto* leaf : leaf_nodes) {
            if (leaf->patient) {
                leaf->patient->update_weight();
                leaf->weight = leaf->patient->weight;
            }
        }
    }

public:
    // Счётчики для анализа эффективности аддитивного свойства
    int ticks_total = 0;
    int rebuilds_global = 0;
    int rebuilds_skipped = 0;

    void printRebuildStats() const {
        cout << "\nСТАТИСТИКА ПЕРЕСТРОЕНИЙ ДЕРЕВА\n";
        cout << "  Всего тактов прогнозирования : " << ticks_total << "\n";
        cout << "  Глобальных перестроений      : " << rebuilds_global <<
"\n";
        cout << "  Пропущено (аддитивность O(1)): " << rebuilds_skipped <<
"\n";
        if (ticks_total > 0) {
            double pct = 100.0 * rebuilds_skipped / ticks_total;
            cout << "  Доля пропущенных перестроений: " << fixed <<
setprecision(1) << pct << "%\n";
            cout << "  Вывод: в " << rebuilds_skipped << " из " <<
ticks_total
                 << " тактов проверка суммарного веса через корень (O(1))\n"
                 << "  позволила избежать глобального перестроения O(n log
n)\n";
        }
    }
```

```
    InverseHuffmanTree() : root(nullptr), patient_count(0) {}
    ~InverseHuffmanTree() { deleteTree(root); }

    void insertPatient(Patient* patient) {
        patient_count++;
        updateAllWeights();
        vector<double> weights; vector<Patient*> patients_list;
        for (auto* leaf : leaf_nodes) {
            if (leaf->patient) { weights.push_back(leaf->patient->weight);
patients_list.push_back(leaf->patient); }
        }
        weights.push_back(patient->weight); patients_list.push_back(patient);
        HuffmanNode* new_root = buildTree(weights, patients_list);
        deleteTree(root); root = new_root;
    }

    Patient* extractMax() {
        if (patient_count == 0) return nullptr;
        updateAllWeights();
        HuffmanNode* max_leaf = nullptr; double max_weight = -1.0;
        for (auto* leaf : leaf_nodes) {
            if (leaf->patient && leaf->weight > max_weight) { max_weight =
leaf->weight; max_leaf = leaf; }
        }
        if (!max_leaf || max_leaf->weight == 0) return nullptr;
        Patient* result = max_leaf->patient;
        vector<double> weights; vector<Patient*> patients_list;
        for (auto* leaf : leaf_nodes) {
            if (leaf != max_leaf && leaf->patient) { weights.push_back(leaf-
>patient->weight); patients_list.push_back(leaf->patient); }
        }
        patient_count--;
        if (patient_count > 0) { HuffmanNode* new_root = buildTree(weights,
patients_list); deleteTree(root); root = new_root; }
        else { deleteTree(root); root = nullptr; leaf_nodes.clear(); }
        return result;
    }

    void removePatient(int id) {
        updateAllWeights();
        HuffmanNode* to_remove = nullptr;
        for (auto* leaf : leaf_nodes) { if (leaf->patient && leaf->patient-
>id == id) { to_remove = leaf; break; } }
        if (!to_remove) { cout << "Пациент с id=" << id << " не найден\n";
return; }
        vector<double> weights; vector<Patient*> patients_list;
        for (auto* leaf : leaf_nodes) {
            if (leaf != to_remove && leaf->patient) { weights.push_back(leaf-
>patient->weight); patients_list.push_back(leaf->patient); }
        }
        patient_count--;
        if (patient_count > 0) { HuffmanNode* new_root = buildTree(weights,
patients_list); deleteTree(root); root = new_root; }
        else { deleteTree(root); root = nullptr; leaf_nodes.clear(); }
        cout << "Пациент id=" << id << " удалён из очереди\n";
    }

    void setPatientRemainingTime(int patient_id, double minutes) {
        for (auto* leaf : leaf_nodes) {
```

```
            if (leaf->patient && leaf->patient->id == patient_id) {
                leaf->patient->set_remaining_time(minutes);
                leaf->weight = leaf->patient->weight;
                updateAncestors(leaf);
                rebuildTree();
                cout << "Пациенту id=" << patient_id << " установлено время "
                    << fixed << setprecision(0) << minutes << " мин\n";
                return;
            }
        }
        cout << "Пациент с id=" << patient_id << " не найден\n";
    }

    void rebuildTree() {
        if (leaf_nodes.empty()) return;
        updateAllWeights();
        vector<double> weights; vector<Patient*> patients_list;
        for (auto* leaf : leaf_nodes) {
            if (leaf->patient) { weights.push_back(leaf->patient->weight);
patients_list.push_back(leaf->patient); }
        }
        HuffmanNode* new_root = buildTree(weights, patients_list);
        deleteTree(root); root = new_root;
    }

    // использование аддитивного свойства.

    void applyPredictionAndRebuild(double dt) {
        if (leaf_nodes.empty()) return;

        // Wcurrent через корень — O(1), аддитивное свойство дерева Хаффмана
        double W_current = getRootWeight();
        if (W_current <= 0) return;

        // Wpred — прогнозируемая сумма весов — O(n), один раз за такт
        double W_pred = 0.0;
        for (auto* leaf : leaf_nodes) {
            if (leaf->patient) W_pred += leaf->patient->predict_weight(dt);
        }

        // Проверка глобального порога через аддитивное свойство — O(1)
        double relative_change = fabs(W_pred - W_current) / W_current;
        ticks_total++;
        if (relative_change > config.THETA) {
            rebuilds_global++;
            // Глобальное перестроение при значимом изменении суммарного веса
            for (auto* leaf : leaf_nodes) {
                if (leaf->patient && leaf->patient->remaining_time > 0) {
                    leaf->patient->weight = leaf->patient-
>predict_weight(dt);
                    leaf->weight = leaf->patient->weight;
                }
                else if (leaf->patient && leaf->patient->remaining_time <= 0)
{
                    leaf->patient->weight = 0;
                    leaf->weight = 0;
                }
            }
            rebuildTree();
```

```
        }
        else {
            rebuilds_skipped++;
            // Суммарный вес изменился незначительно — только локальные
обновления
            bool any_local = false;
            for (auto* leaf : leaf_nodes) {
                if (!leaf->patient) continue;
                if (leaf->patient->remaining_time <= 0 && leaf->patient-
>weight != 0) {
                    leaf->patient->weight = 0;
                    leaf->weight = 0;
                    updateAncestors(leaf);
                    any_local = true;
                }
                else if (leaf->patient->needs_rebuild(dt)) {
                    leaf->patient->weight = leaf->patient-
>predict_weight(dt);
                    leaf->weight = leaf->patient->weight;
                    updateAncestors(leaf);  // локальное обновление предков
O(log n)
                    any_local = true;
                }
            }
            (void)any_local;
        }
    }

    double getRootWeight() const { return root ? root->weight : 0.0; }
    int size() const { return patient_count; }
    bool empty() const { return patient_count == 0; }

    vector<Patient*> getAllPatients() {
        vector<Patient*> result;
        for (auto* leaf : leaf_nodes) { if (leaf->patient)
result.push_back(leaf->patient); }
        return result;
    }

    Patient* findPatient(int id) {
        for (auto* leaf : leaf_nodes) { if (leaf->patient && leaf->patient-
>id == id) return leaf->patient; }
        return nullptr;
    }
};

// СРАВНЕНИЕ СТРУКТУР ДАННЫХ
// Реализованы три структуры из главы 1.2 курсовой:
// 1. Несортированный список (O(1) вставка, O(n) извлечение)
// 2. Сортированный список  (O(n) вставка, O(1) извлечение)
// 3. Бинарная куча STL     (O(log n) обе операции)

// Несортированный список
struct UnsortedListQueue {
    vector<pair<double, int>> data; // (вес, id)
    void push(double w, int id) { data.push_back({ w, id }); }
    pair<double, int> extractMax() {
        auto it = max_element(data.begin(), data.end());
        auto result = *it; data.erase(it); return result;
```

```
    }
    int size() const { return data.size(); }
};

// Сортированный список (поддерживается в порядке убывания)
struct SortedListQueue {
    list<pair<double, int>> data;
    void push(double w, int id) {
        auto it = data.begin();
        while (it != data.end() && it->first > w) ++it;
        data.insert(it, { w, id });
    }
    pair<double, int> extractMax() {
        auto result = data.front(); data.pop_front(); return result;
    }
    int size() const { return data.size(); }
};

// Бинарная куча STL
struct BinaryHeapQueue {
    priority_queue<pair<double, int>> data;
    void push(double w, int id) { data.push({ w, id }); }
    pair<double, int> extractMax() { auto r = data.top(); data.pop(); return
r; }
    int size() const { return data.size(); }
};

// Результат бенчмарка
struct BenchmarkResult {
    string name;
    double push_ms;
    double extract_ms;
    double total_ms;
    int n;
};
// Функция запуска сравнительного теста для заданного n
// Тестирует push и extractMax для всех трёх структур
BenchmarkResult runBenchmark(const string& name, int n,
    function<void(double, int)> push_fn,
    function<void()> extract_fn)
{
    BenchmarkResult res;
    res.name = name; res.n = n;

    // Генерация весов
    vector<double> weights(n);
    uniform_real_distribution<> w_dist(1e-6, 1.0);
    for (auto& w : weights) w = w_dist(rng);

    // Тест вставки
    Timer t1;
    for (int i = 0; i < n; ++i) push_fn(weights[i], i);
    res.push_ms = t1.elapsed_ms();

    // Тест извлечения
    Timer t2;
    for (int i = 0; i < n; ++i) extract_fn();
    res.extract_ms = t2.elapsed_ms();
```

```
    res.total_ms = res.push_ms + res.extract_ms;
    return res;
}

void runComparison(int n) {
    cout << "\nСРАВНЕНИЕ СТРУКТУР ДАННЫХ (n = " << n << " пациентов)\n";
    cout << left << setw(25) << "Структура"
        << right << setw(15) << "Вставка (мс)"
        << setw(15) << "Извлеч. (мс)"
        << setw(15) << "Итого (мс)" << "\n";

    vector<BenchmarkResult> results;

    // 1. Несортированный список
    {
        UnsortedListQueue q;
        auto res = runBenchmark("Несорт. список", n,
            [&](double w, int id) { q.push(w, id); },
            [&]() { q.extractMax(); });
        results.push_back(res);
    }

    // 2. Сортированный список
    {
        SortedListQueue q;
        auto res = runBenchmark("Сортир. список", n,
            [&](double w, int id) { q.push(w, id); },
            [&]() { q.extractMax(); });
        results.push_back(res);
    }

    // 3. Бинарная куча STL
    {
        BinaryHeapQueue q;
        auto res = runBenchmark("Бинарная куча", n,
            [&](double w, int id) { q.push(w, id); },
            [&]() { q.extractMax(); });
        results.push_back(res);
    }

    // 4. Инверсное дерево Хаффмана (только построение + извлечение)
    {
        // Генерируем минимальных пациентов для теста
        vector<Patient*> patients(n);
        for (int i = 0; i < n; ++i) {
            patients[i] = new Patient();
            patients[i]->id = i;
            patients[i]->remaining_time = 600 + i * 10;
            patients[i]->treatment_time = 1200;
            patients[i]->weight = 1.0 / (patients[i]->remaining_time *
patients[i]->treatment_time);
        }

        BenchmarkResult res;
        res.name = "Инв. д. Хаффмана"; res.n = n;

        InverseHuffmanTree tree;
        Timer t1;
        for (int i = 0; i < n; ++i) tree.insertPatient(patients[i]);
```

```
        res.push_ms = t1.elapsed_ms();

        Timer t2;
        for (int i = 0; i < n; ++i) {
            Patient* p = tree.extractMax();
            if (!p) break;
        }
        res.extract_ms = t2.elapsed_ms();
        res.total_ms = res.push_ms + res.extract_ms;
        results.push_back(res);

        for (auto* p : patients) delete p;
    }

    for (auto& r : results) {
        cout << left << setw(25) << r.name
            << right << fixed << setprecision(3)
            << setw(15) << r.push_ms
            << setw(15) << r.extract_ms
            << setw(15) << r.total_ms << "\n";
    }
    cout << "\n";
}

struct CSVRecord {
    int level, age, pain_level, chronic_diseases, prev_visits;
    double heart_rate, systolic_bp, oxygen_saturation, temperature;
    string arrival_mode;
};

vector<CSVRecord> loadAllCSV(const string& filename) {
    vector<CSVRecord> records;
    ifstream file(filename);
    if (!file.is_open()) return records;
    string line;
    getline(file, line);
    while (getline(file, line)) {
        if (line.empty()) continue;
        stringstream ss(line); CSVRecord rec; string token;
        getline(ss, token, ','); rec.age = stoi(token);
        getline(ss, token, ','); rec.heart_rate = stod(token);
        getline(ss, token, ','); rec.systolic_bp = stod(token);
        getline(ss, token, ','); rec.oxygen_saturation = stod(token);
        getline(ss, token, ','); rec.temperature = stod(token);
        getline(ss, token, ','); rec.pain_level = stoi(token);
        getline(ss, token, ','); rec.chronic_diseases = stoi(token);
        getline(ss, token, ','); rec.prev_visits = stoi(token);
        getline(ss, token, ','); rec.arrival_mode = token;
        getline(ss, token, ','); rec.level = stoi(token);
        records.push_back(rec);
    }
    return records;
}

bool show_table = true;

void printQueueTable(const vector<Patient*>& patients) {
    if (!show_table) return;
    vector<Patient*> active, expired;
```

```
    for (auto* p : patients) { if (p->weight > 0) active.push_back(p); else
expired.push_back(p); }
    sort(active.begin(), active.end(), [](Patient* a, Patient* b) { return a-
>weight > b->weight; });
    cout << "\n" << string(95, '-') << "\n";
    cout << "| " << setw(4) << "ID" << " | " << setw(5) << "УРОВ" << " | " <<
setw(14) << "ВЕС"
        << " | " << setw(10) << "ОЖИД(мин)" << " | " << setw(10) <<
"ОСТ(мин)"
        << " | " << setw(10) << "АЛЬФА" << " |" << endl;
    cout << string(95, '-') << "\n";
    for (auto* p : active) {
        cout << "| " << setw(4) << p->id << " | " << setw(5) << p->level
            << " | " << fixed << setw(14) << setprecision(8) << p->weight
            << " | " << fixed << setw(10) << setprecision(1) << p->wait_time
/ 60.0
            << " | " << setw(10) << setprecision(1) << p->remaining_time /
60.0
            << " | " << setw(10) << setprecision(3) << p->alpha << " |" <<
endl;
    }
    for (auto* p : expired) {
        cout << "| " << setw(4) << p->id << " | " << setw(5) << p->level
            << " | " << setw(14) << "0.00000000"
            << " | " << setw(10) << setprecision(1) << p->wait_time / 60.0
            << " | " << setw(10) << setprecision(1) << "0.0"
            << " | " << setw(10) << setprecision(3) << p->alpha << " |
[ИСТЕКЛО]" << endl;
    }
    cout << string(95, '-') << "\n";
    cout << "Всего в очереди: " << patients.size()
        << " (активных: " << active.size() << ", истекших: " <<
expired.size() << ")" << endl;
}

class Simulator {
private:
    InverseHuffmanTree tree;
    vector<CSVRecord> all_csv_data;
    vector<Patient*> all_loaded_patients;
    int next_csv_index;
    double current_time;
    int next_patient_id;

    int getNextId() { return next_patient_id++; }

    void addPatientFromCSV(int csv_index) {
        if (csv_index >= (int)all_csv_data.size()) {
            cout << "Нет пациента с индексом " << csv_index << " в CSV (всего
записей: " << all_csv_data.size() << ")\n";
            return;
        }
        const auto& rec = all_csv_data[csv_index];
        Patient* p = new Patient();
        p->init_from_csv(getNextId(), rec.level, rec.age, rec.heart_rate,
rec.systolic_bp,
            rec.oxygen_saturation, rec.temperature, rec.pain_level,
            rec.chronic_diseases, rec.prev_visits, rec.arrival_mode);
        tree.insertPatient(p);
```

```
        all_loaded_patients.push_back(p);
        cout << "\n[ДОБАВЛЕН] Пациент id=" << p->id << ", уровень=" << p-
>level
            << ", вес=" << fixed << setprecision(8) << p->weight
            << ", возраст=" << p->age << " лет" << endl;
    }

    void autoPredict() {
        if (config.auto_predict && !tree.empty())
            tree.applyPredictionAndRebuild(config.DT);
    }

public:
    Simulator() : next_csv_index(0), current_time(0), next_patient_id(0) {
srand(time(nullptr)); }
    ~Simulator() { for (auto* p : all_loaded_patients) delete p; }

    void autoLoadFirstPatients(const string& filename, int count = 50) {
        all_csv_data = loadAllCSV(filename);
        if (all_csv_data.empty()) {
            cout << "CSV-файл не найден или пуст. Генерация синтетических
пациентов...\n";
            uniform_int_distribution<> level_dist(0, 3), age_dist(18, 90),
pain_dist(0, 10), chronic_dist(0, 5), prev_dist(0, 5), mode_dist(0, 1);
            uniform_real_distribution<> hr_dist(60, 120), bp_dist(100, 160),
spo2_dist(88, 100), temp_dist(36.0, 39.0);
            for (int i = 0; i < count; ++i) {
                CSVRecord rec;
                rec.level = level_dist(rng); rec.age = age_dist(rng);
                rec.heart_rate = hr_dist(rng); rec.systolic_bp =
bp_dist(rng);
                rec.oxygen_saturation = spo2_dist(rng); rec.temperature =
temp_dist(rng);
                rec.pain_level = pain_dist(rng); rec.chronic_diseases =
chronic_dist(rng);
                rec.prev_visits = prev_dist(rng);
                rec.arrival_mode = (mode_dist(rng) == 0) ? "walk_in" :
"ambulance";
                all_csv_data.push_back(rec);
            }
        }
        cout << "Загружено " << all_csv_data.size() << " записей. Добавляем
первых "
            << min(count, (int)all_csv_data.size()) << " пациентов в
очередь...\n";
        for (int i = 0; i < min(count, (int)all_csv_data.size()); ++i)
addPatientFromCSV(i);
        next_csv_index = min(count, (int)all_csv_data.size());
    }

    void addNextFromCSV() {
        if (all_csv_data.empty()) { cout << "CSV не загружен\n"; return; }
        if (next_csv_index >= (int)all_csv_data.size()) { cout << "Все
пациенты из CSV уже добавлены\n"; return; }
        addPatientFromCSV(next_csv_index++);
        autoPredict();
    }

    void addById(int csv_index) {
```

```
        if (all_csv_data.empty()) { cout << "CSV не загружен\n"; return; }
        if (csv_index < 0 || csv_index >= (int)all_csv_data.size()) {
            cout << "Индекс должен быть от 0 до " << all_csv_data.size() - 1
<< endl; return;
        }
        addPatientFromCSV(csv_index);
        autoPredict();
    }

    void takePatient() {
        Patient* next = tree.extractMax();
        if (!next) { cout << "Очередь пуста или все пациенты с истекшим
временем\n"; return; }
        double wait_time = current_time - next->arrival_time;
        cout << "\n[ПРИНЯТ] Пациент id=" << next->id << ", уровень=" << next-
>level
            << ", ожидал=" << fixed << setprecision(1) << wait_time / 60.0 <<
" мин"
            << ", вес=" << fixed << setprecision(8) << next->weight << endl;
        auto it = find(all_loaded_patients.begin(),
all_loaded_patients.end(), next);
        if (it != all_loaded_patients.end()) all_loaded_patients.erase(it);
        delete next;
        autoPredict();
    }

    void removePatient(int id) {
        Patient* p = tree.findPatient(id);
        if (!p) { cout << "Пациент с id=" << id << " не найден в очереди\n";
return; }
        tree.removePatient(id);
        auto it = find(all_loaded_patients.begin(),
all_loaded_patients.end(), p);
        if (it != all_loaded_patients.end()) all_loaded_patients.erase(it);
        delete p;
        autoPredict();
    }

    void setPatientRemainingTime(int id, double minutes) {
        tree.setPatientRemainingTime(id, minutes);
        autoPredict();
    }

    void worsenPatient(int id) {
        Patient* p = tree.findPatient(id);
        if (!p) { cout << "Пациент с id=" << id << " не найден в очереди\n";
return; }
        cout << "\n[УХУДШЕНИЕ] Пациент id=" << id << endl;
        p->worsen_random();
        cout << "  Новый вес: " << fixed << setprecision(8) << p->weight <<
endl;
        if (p->needs_rebuild(config.DT)) {
            cout << "  Изменение веса превысило порог " << config.THETA * 100
<< "%. Перестраиваем дерево." << endl;
            tree.rebuildTree();
        }
        else {
            cout << "  Изменение веса в пределах порога. Дерево не
перестраивается." << endl;
```

```
        }
    }

    void stepTime(double minutes) {
        if (minutes < 0) { cout << "Ошибка: время не может быть
отрицательным\n"; return; }
        Timer t;
        double delta_sec = minutes * 60.0;
        current_time += delta_sec;
        auto patients = tree.getAllPatients();
        for (auto* p : patients) p->update_wait_time(delta_sec);
        tree.rebuildTree();
        cout << "\n[ВРЕМЯ] Прошло " << fixed << setprecision(1) << minutes <<
" мин";
        cout << " (время выполнения: " << fixed << setprecision(3) <<
t.elapsed_ms() << " мс)" << endl;
        autoPredict();
    }

    void showQueue() {
        Timer t;
        auto patients = tree.getAllPatients();
        printQueueTable(patients);
        cout << " (время выполнения: " << fixed << setprecision(3) <<
t.elapsed_ms() << " мс)" << endl;
    }

    void showPatientInfo(int id) {
        Timer t;
        Patient* p = tree.findPatient(id);
        if (!p) { cout << "Пациент с id=" << id << " не найден в очереди\n";
return; }
        p->print_info();
        cout << " (время выполнения: " << fixed << setprecision(3) <<
t.elapsed_ms() << " мс)" << endl;
    }

    void printHelp() {
        cout << "\nДОСТУПНЫЕ КОМАНДЫ:\n";
        cout << "  addnext                    - добавить следующего пациента
из CSV\n";
        cout << "  add_id <индекс>            - добавить пациента из CSV по
индексу\n";
        cout << "  take                       - принять следующего
пациента\n";
        cout << "  remove <id>                - удалить пациента из
очереди\n";
        cout << "  set_rem <id> <мин>         - установить оставшееся
безопасное время\n";
        cout << "  worsen <id>                - ухудшение состояния
пациента\n";
        cout << "  step <мин>                 - продвинуть время (в
минутах)\n";
        cout << "  list                       - показать очередь\n";
        cout << "  info <id>                  - показать данные по
пациенту\n";
        cout << "  benchmark                  - сравнение структур
данных\n";
```

```
        cout << "  stats                          - статистика перестроений
дерева\n";
        cout << "  help                           - показать эту справку\n";
        cout << "  exit                           - выход\n";
    }

    void runInteractive() {
        cout << "\nСимулятор запущен. Введите 'help' для списка команд.\n";
        string cmd;
        while (true) {
            cout << "> ";
            getline(cin, cmd);
            if (cmd.empty()) continue;
            stringstream ss(cmd); string action; ss >> action;

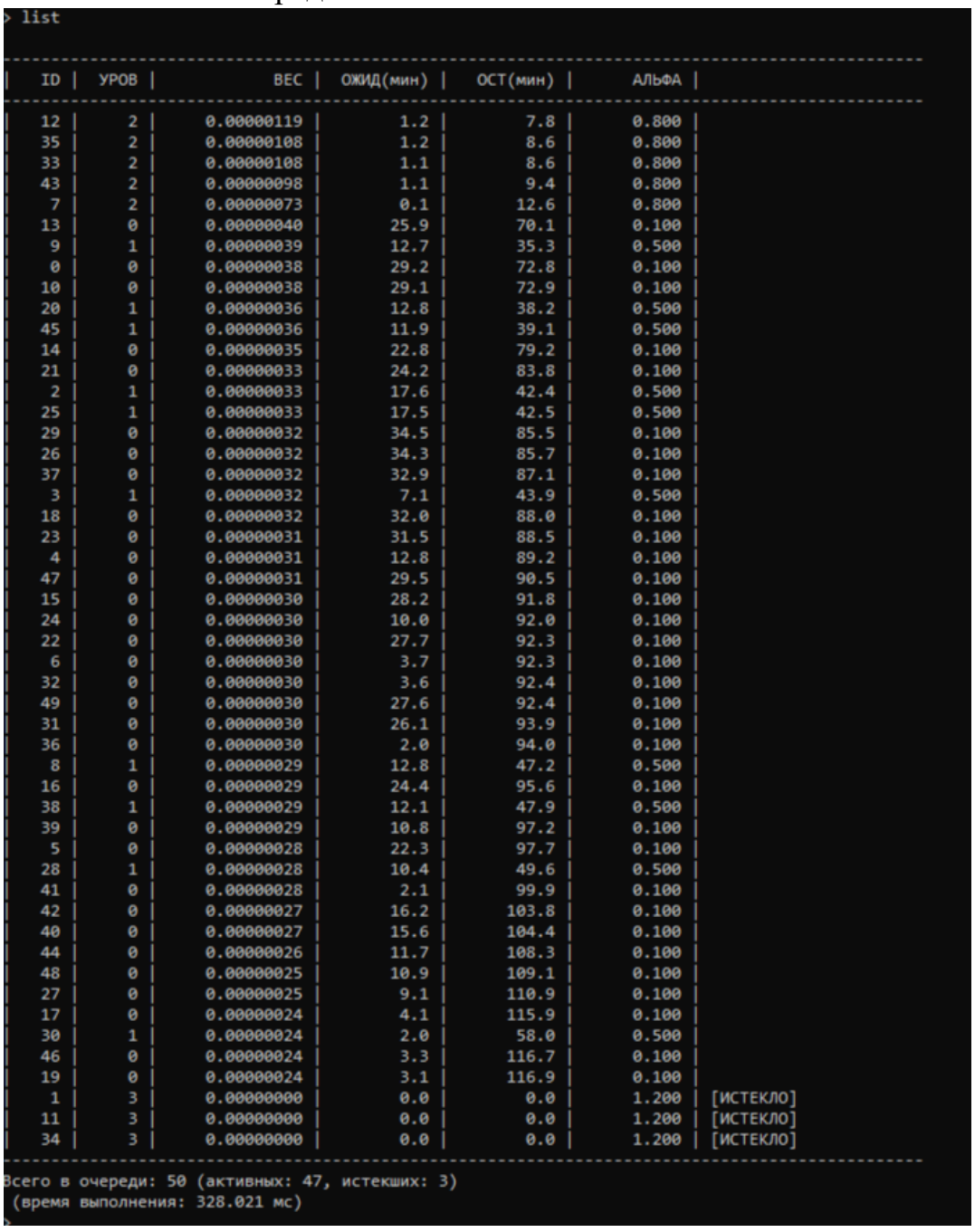
            if (action == "addnext") { Timer t; addNextFromCSV(); cout << "
(время выполнения: " << fixed << setprecision(3) << t.elapsed_ms() << " мс)"
<< endl; }
            else if (action == "add_id") { int idx; if (ss >> idx) { Timer t;
addById(idx); cout << " (время выполнения: " << fixed << setprecision(3) <<
t.elapsed_ms() << " мс)" << endl; } else cout << "Использование: add_id
<индекс>\n"; }
            else if (action == "take") { Timer t; takePatient(); cout << "
(время выполнения: " << fixed << setprecision(3) << t.elapsed_ms() << " мс)"
<< endl; }
            else if (action == "remove") { int id; if (ss >> id) { Timer t;
removePatient(id); cout << " (время выполнения: " << fixed << setprecision(3)
<< t.elapsed_ms() << " мс)" << endl; } else cout << "Использование: remove
<id>\n"; }
            else if (action == "set_rem") { int id; double minutes; if (ss >>
id >> minutes) { Timer t; setPatientRemainingTime(id, minutes); cout << "
(время выполнения: " << fixed << setprecision(3) << t.elapsed_ms() << " мс)"
<< endl; } else cout << "Использование: set_rem <id> <минуты>\n"; }
            else if (action == "worsen") { int id; if (ss >> id) { Timer t;
worsenPatient(id); cout << " (время выполнения: " << fixed << setprecision(3)
<< t.elapsed_ms() << " мс)" << endl; } else cout << "Использование: worsen
<id>\n"; }
            else if (action == "step") { double minutes; if (ss >> minutes)
stepTime(minutes); else cout << "Использование: step <минуты>\n"; }
            else if (action == "list") showQueue();
            else if (action == "info") { int id; if (ss >> id)
showPatientInfo(id); else cout << "Использование: info <id>\n"; }
            else if (action == "benchmark") {
                for (int n : {10, 50, 100}) runComparison(n);
            }
            else if (action == "stats") {
                tree.printRebuildStats();
            }
            else if (action == "help") printHelp();
            else if (action == "exit" || action == "quit") { cout <<
"Выход...\n"; break; }
            else cout << "Неизвестная команда. Введите 'help'\n";
        }
    }
};

int main() {
    setlocale(LC_ALL, "Russian");
    cout << "ДИНАМИЧЕСКОЕ ДЕРЕВО ХАФФМАНА\n";
```

```
    cout << "Симуляция очереди приёмного отделения\n";
    Simulator sim;
    sim.autoLoadFirstPatients("synthetic_medical_triage.csv", 50);
    sim.runInteractive();
    return 0;
}
```

Приложение 10.

Тест 1. Список очереди



```
> list

-------------------------------------------------------------------------------------------
|   ID |  УРОВ |          ВЕС |  ОЖИД(мин) |  ОСТ(мин) |      АЛЬФА |
-------------------------------------------------------------------------------------------
|   12 |     2 |    0.00000119 |        1.2 |       7.8 |       0.800 |
|   35 |     2 |    0.00000108 |        1.2 |       8.6 |       0.800 |
|   33 |     2 |    0.00000108 |        1.1 |       8.6 |       0.800 |
|   43 |     2 |    0.00000098 |        1.1 |       9.4 |       0.800 |
|    7 |     2 |    0.00000073 |        0.1 |      12.6 |       0.800 |
|   13 |     0 |    0.00000040 |       25.9 |      70.1 |       0.100 |
|    9 |     1 |    0.00000039 |       12.7 |      35.3 |       0.500 |
|    0 |     0 |    0.00000038 |       29.2 |      72.8 |       0.100 |
|   10 |     0 |    0.00000038 |       29.1 |      72.9 |       0.100 |
|   20 |     1 |    0.00000036 |       12.8 |      38.2 |       0.500 |
|   45 |     1 |    0.00000036 |       11.9 |      39.1 |       0.500 |
|   14 |     0 |    0.00000035 |       22.8 |      79.2 |       0.100 |
|   21 |     0 |    0.00000033 |       24.2 |      83.8 |       0.100 |
|    2 |     1 |    0.00000033 |       17.6 |      42.4 |       0.500 |
|   25 |     1 |    0.00000033 |       17.5 |      42.5 |       0.500 |
|   29 |     0 |    0.00000032 |       34.5 |      85.5 |       0.100 |
|   26 |     0 |    0.00000032 |       34.3 |      85.7 |       0.100 |
|   37 |     0 |    0.00000032 |       32.9 |      87.1 |       0.100 |
|    3 |     1 |    0.00000032 |        7.1 |      43.9 |       0.500 |
|   18 |     0 |    0.00000032 |       32.0 |      88.0 |       0.100 |
|   23 |     0 |    0.00000031 |       31.5 |      88.5 |       0.100 |
|    4 |     0 |    0.00000031 |       12.8 |      89.2 |       0.100 |
|   47 |     0 |    0.00000031 |       29.5 |      90.5 |       0.100 |
|   15 |     0 |    0.00000030 |       28.2 |      91.8 |       0.100 |
|   24 |     0 |    0.00000030 |       10.0 |      92.0 |       0.100 |
|   22 |     0 |    0.00000030 |       27.7 |      92.3 |       0.100 |
|    6 |     0 |    0.00000030 |        3.7 |      92.3 |       0.100 |
|   32 |     0 |    0.00000030 |        3.6 |      92.4 |       0.100 |
|   49 |     0 |    0.00000030 |       27.6 |      92.4 |       0.100 |
|   31 |     0 |    0.00000030 |       26.1 |      93.9 |       0.100 |
|   36 |     0 |    0.00000030 |        2.0 |      94.0 |       0.100 |
|    8 |     1 |    0.00000029 |       12.8 |      47.2 |       0.500 |
|   16 |     0 |    0.00000029 |       24.4 |      95.6 |       0.100 |
|   38 |     1 |    0.00000029 |       12.1 |      47.9 |       0.500 |
|   39 |     0 |    0.00000029 |       10.8 |      97.2 |       0.100 |
|    5 |     0 |    0.00000028 |       22.3 |      97.7 |       0.100 |
|   28 |     1 |    0.00000028 |       10.4 |      49.6 |       0.500 |
|   41 |     0 |    0.00000028 |        2.1 |      99.9 |       0.100 |
|   42 |     0 |    0.00000027 |       16.2 |     103.8 |       0.100 |
|   40 |     0 |    0.00000027 |       15.6 |     104.4 |       0.100 |
|   44 |     0 |    0.00000026 |       11.7 |     108.3 |       0.100 |
|   48 |     0 |    0.00000025 |       10.9 |     109.1 |       0.100 |
|   27 |     0 |    0.00000025 |        9.1 |     110.9 |       0.100 |
|   17 |     0 |    0.00000024 |        4.1 |     115.9 |       0.100 |
|   30 |     1 |    0.00000024 |        2.0 |      58.0 |       0.500 |
|   46 |     0 |    0.00000024 |        3.3 |     116.7 |       0.100 |
|   19 |     0 |    0.00000024 |        3.1 |     116.9 |       0.100 |
|    1 |     3 |    0.00000000 |        0.0 |       0.0 |       1.200 | [ИСТЕКЛО]
|   11 |     3 |    0.00000000 |        0.0 |       0.0 |       1.200 | [ИСТЕКЛО]
|   34 |     3 |    0.00000000 |        0.0 |       0.0 |       1.200 | [ИСТЕКЛО]
-------------------------------------------------------------------------------------------
Всего в очереди: 50 (активных: 47, истекших: 3)
(время выполнения: 328.021 мс)
```

Тест 2. Добавление пациента

```
> addnext

[ДОБАВЛЕН] Пациент id=51, уровень=1, вес=0.00000025, возраст=7 лет
 (время выполнения: 11.623 мс)
> list
```

ID	УРОВ	ВЕС	ОЖИД(мин)	ОСТ(мин)	АЛЬФА
12	2	0.00000119	1.2	7.8	0.800
35	2	0.00000108	1.2	8.6	0.800
33	2	0.00000108	1.1	8.6	0.800
43	2	0.00000098	1.1	9.4	0.800
7	2	0.00000073	0.1	12.6	0.800
13	0	0.00000040	25.9	70.1	0.100
9	1	0.00000039	12.7	35.3	0.500
0	0	0.00000038	29.2	72.8	0.100
10	0	0.00000038	29.1	72.9	0.100
20	1	0.00000036	12.8	38.2	0.500
45	1	0.00000036	11.9	39.1	0.500
14	0	0.00000035	22.8	79.2	0.100
21	0	0.00000033	24.2	83.8	0.100
2	1	0.00000033	17.6	42.4	0.500
25	1	0.00000033	17.5	42.5	0.500
29	0	0.00000032	34.5	85.5	0.100
26	0	0.00000032	34.3	85.7	0.100
37	0	0.00000032	32.9	87.1	0.100
3	1	0.00000032	7.1	43.9	0.500
18	0	0.00000032	32.0	88.0	0.100
23	0	0.00000031	31.5	88.5	0.100
4	0	0.00000031	12.8	89.2	0.100
47	0	0.00000031	29.5	90.5	0.100
15	0	0.00000030	28.2	91.8	0.100
24	0	0.00000030	10.0	92.0	0.100
22	0	0.00000030	27.7	92.3	0.100
6	0	0.00000030	3.7	92.3	0.100
32	0	0.00000030	3.6	92.4	0.100
49	0	0.00000030	27.6	92.4	0.100
31	0	0.00000030	26.1	93.9	0.100
36	0	0.00000030	2.0	94.0	0.100
8	1	0.00000029	12.8	47.2	0.500
16	0	0.00000029	24.4	95.6	0.100
38	1	0.00000029	12.1	47.9	0.500
39	0	0.00000029	10.8	97.2	0.100
5	0	0.00000028	22.3	97.7	0.100
28	1	0.00000028	10.4	49.6	0.500
41	0	0.00000028	2.1	99.9	0.100
42	0	0.00000027	16.2	103.8	0.100
40	0	0.00000027	15.6	104.4	0.100
44	0	0.00000026	11.7	108.3	0.100
48	0	0.00000025	10.9	109.1	0.100
27	0	0.00000025	9.1	110.9	0.100
51	1	0.00000025	4.2	55.8	0.500

Тест 3. Добавление по индексу из CSV

```
 add_id 77

ДОБАВЛЕН] Пациент id=52, уровень=0, вес=0.00000026, возраст=21 лет
(время выполнения: 4.151 мс)
```

Тест 4. Извлечение пациента

```
> take

[ПРИНЯТ] Пациент id=9, уровень=1, ожидал=42.7 мин, вес=0.00000261
 (время выполнения: 3.554 мс)
>
```

Тест 5. Ухудшение состояния

```
> worsen 30

[УХУДШЕНИЕ] Пациент id=30
  Новый вес: 0.00000023
  Изменение веса превысило порог 15.00000000%. Перестраиваем дерево.
 (время выполнения: 11.673 мс)
```

Тест 6. Установка безопасного времени истекшему пациенту

```
> set_rem 40
Использование: set_rem <id> <минуты>
> set_rem 40 60
Пациенту id=40 установлено время 60 мин
 (время выполнения: 2.606 мс)
```

Тест 7. Продвижение времени

```
> step 35

[ВРЕМЯ] Прошло 35.0 мин (время выполнения: 1.884 мс)
```

Тест 8. Обработка ошибки (отрицательное время)

```
> step -10
Ошибка: время не может быть отрицательным
>
```

Тест 9. Обработка ошибки (несуществующий индекс)

```
> info 99
Пациент с id=99 не найден в очереди
> info 4949512
Пациент с id=4949512 не найден в очереди
> add_id 5904040
Индекс должен быть от 0 до 17999
 (время выполнения: 2.071 мс)
>
```

Тест 10. Информация о пациенте

```
> info 32
ИНФОРМАЦИЯ О ПАЦИЕНТЕ id=32Уровень триажа: 0
Возраст: 36 лет
Пульс: 84 уд/мин
Давление: 112 мм рт.ст.
Сатурация: 98%
Температура: 37.0°C
Уровень боли: 2/10
Хронические заболевания: 0
Предыдущие обращения: 1
Способ поступления: ambulance
Критичность S(t): 36.67
Оставшееся безопасное время: 27.41 мин
Ожидаемое время лечения: 10.00 мин
Коэффициент альфа: 0.10
Коэффициент бета: 0.00
Текущий вес: 0.00000101
Время ожидания: 68.6 мин (время выполнения: 27.983 мс)
>
```

Тест 11. Сравнение структур данных

```
> benchmark

СРАВНЕНИЕ СТРУКТУР ДАННЫХ (n = 10 пациентов)
Структура                  Вставка (мс)   Извлеч. (мс)     Итого (мс)
Несорт. список                    0.025          0.030          0.056
Сортир. список                    0.028          0.006          0.034
Бинарная куча                     0.021          0.012          0.033
Инв. д. Хаффмана                  0.330          0.263          0.593


СРАВНЕНИЕ СТРУКТУР ДАННЫХ (n = 50 пациентов)
Структура                  Вставка (мс)   Извлеч. (мс)     Итого (мс)
Несорт. список                    0.033          0.185          0.218
Сортир. список                    0.238          0.023          0.262
Бинарная куча                     0.061          0.064          0.125
Инв. д. Хаффмана                  4.635          3.957          8.592


СРАВНЕНИЕ СТРУКТУР ДАННЫХ (n = 100 пациентов)
Структура                  Вставка (мс)   Извлеч. (мс)     Итого (мс)
Несорт. список                    0.063          0.739          0.802
Сортир. список                    0.932          0.051          0.983
Бинарная куча                     0.127          0.162          0.289
Инв. д. Хаффмана                 18.801         15.858         34.658
```

Тест 12. Статистика перестроений

```
> step 1

[ВРЕМЯ] Прошло 1.0 мин (время выполнения: 3.733 мс)
> step 1

[ВРЕМЯ] Прошло 1.0 мин (время выполнения: 3.390 мс)
> step 1

[ВРЕМЯ] Прошло 1.0 мин (время выполнения: 3.440 мс)
> step 1

[ВРЕМЯ] Прошло 1.0 мин (время выполнения: 4.121 мс)
> stats

СТАТИСТИКА ПЕРЕСТРОЕНИЙ ДЕРЕВА
  Всего тактов прогнозирования : 4
  Глобальных перестроений      : 0
  Пропущено (аддитивность O(1)): 4
  Доля пропущенных перестроений: 100.0%
  Вывод: в 4 из 4 тактов проверка суммарного веса через корень (O(1))
  позволила избежать глобального перестроения O(n log n)
```